

SOCIAL MEDIA AND WEB ANALYTICS

Prof Dr Matthias Bogaert

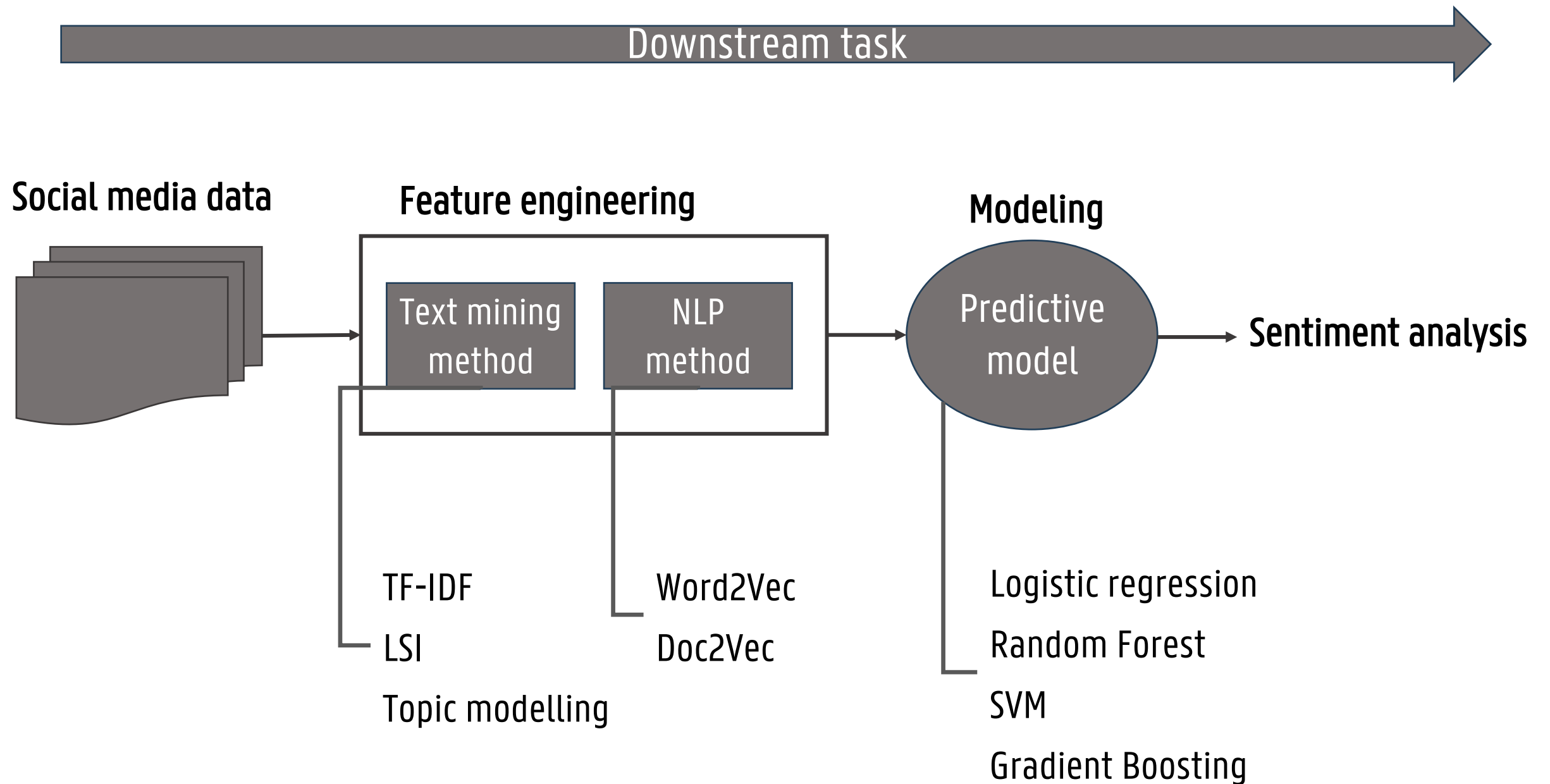
LECTURE 5: SENTIMENT ANALYSIS WITH LLMS

CONTENT

- Lecture 5: Sentiment analysis with Large Language Models (LLMs)
 - Modern NLP approaches
 - Transformer models
 - Encoder-models for sentiment classification
 - Prompt engineering for sentiment analysis
 - Revisiting topic modeling with BERTopic

MODERN NLP APPROACHES

APPROACH: PREVIOUS LECTURE



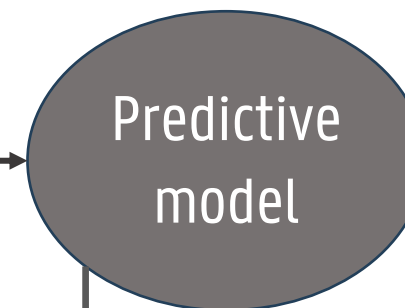
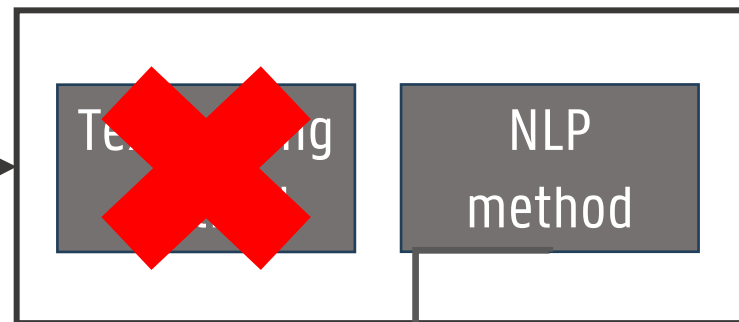
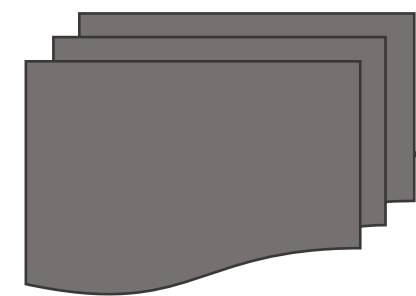
APPROACH: DEEP LEARNING

Downstream task

Social media data

Feature engineering

Modeling



Sentiment analysis

Pre-trained
embeddings

Deep Neural Network
Recurrent Neural Network
Convolutional Neural Network

Deep Neural Network
Recurrent Neural Network
Convolutional Neural Network

APPROACH: DEEP LEARNING

— Deep learning

— Input

- Pre-trained word embeddings
- Task-learning : raw text tokens, so less preprocessing required (e.g., text should not be stemmed or lemmatized) and embeddings are learned jointly with your task inside your neural network (in the same way as your weights).
- Word embeddings are flattened (DNN) or pooled (CNN) or handled in sequence (RNN)
- Paragraph vectors can also be used in a DNN

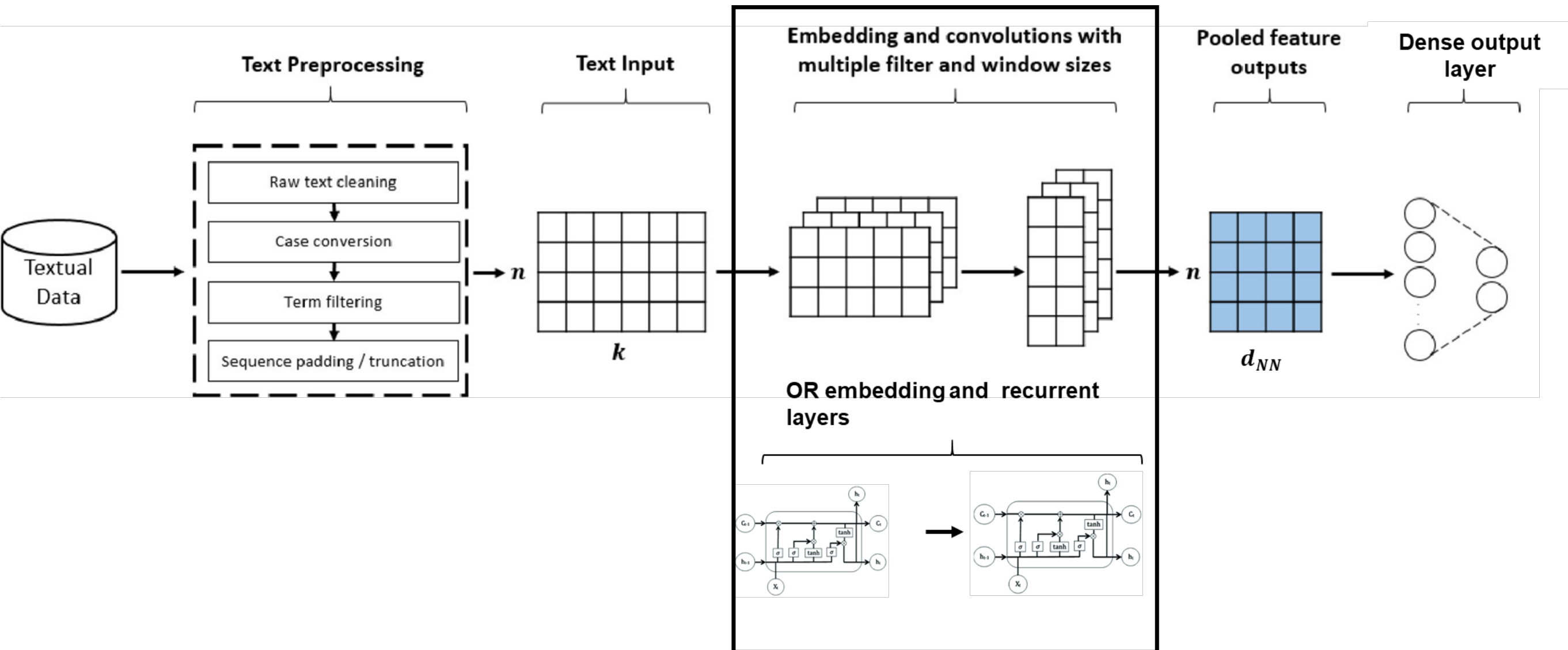
— Models

- Deep neural network (i.e., fully connected neural network with more hidden layers)
- Convolutional neural networks
- Recurrent neural networks
 - Traditional RNN
 - Long-short-term memory (LSTM)
 - Gated recurrent unit (GRU)

- Performance: GRUs, LSTMs and CNNs are performant
- Careful hyper-parameter tuning is crucial!

APPOACH: DEEP LEARNING

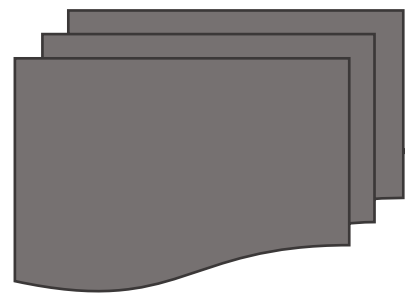
— Deep learning set-up



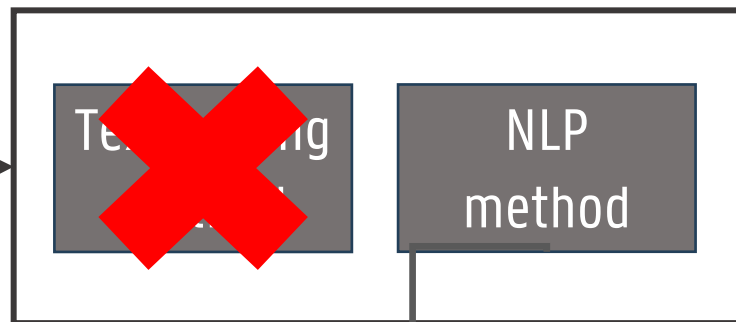
APPROACH: TRANSFORMERS

Downstream task

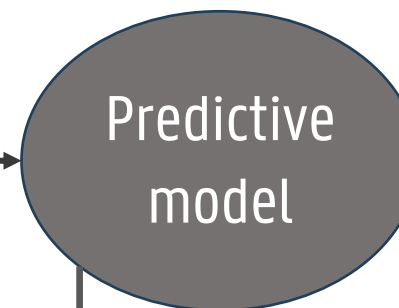
Social media data



Feature engineering



Modeling



Sentiment analysis

Pre-trained
Encoder-based
embeddings

Logistic regression
Random Forest
SVM
Gradient Boosting

Fine-tuned Encoder models
Prompt-based Decoder models

TRANSFORMER MODELS

WHY TRANSFORMERS?

- Traditional approaches effectively treat text as a Bag-of-Words or fixed vectors
- **Problem 1:** They ignore word order
 - *"This movie is good, not bad" vs "This movie is bad, not good"*
 - To TF-IDF or Doc2Vec these look identical (same words, same counts), but sentiment is opposite
- **Problem 2:** They are context independent
 - Word2Vec assigns one fixed vector per word, regardless of meaning
 - *"The dog is sick" vs "That beat was sick!"*
 - Static models average these meanings, confusing the classifier
- We need a model that understands **context** and **sequence**, not just the word presence and isolated meaning

WHY TRANSFORMERS?

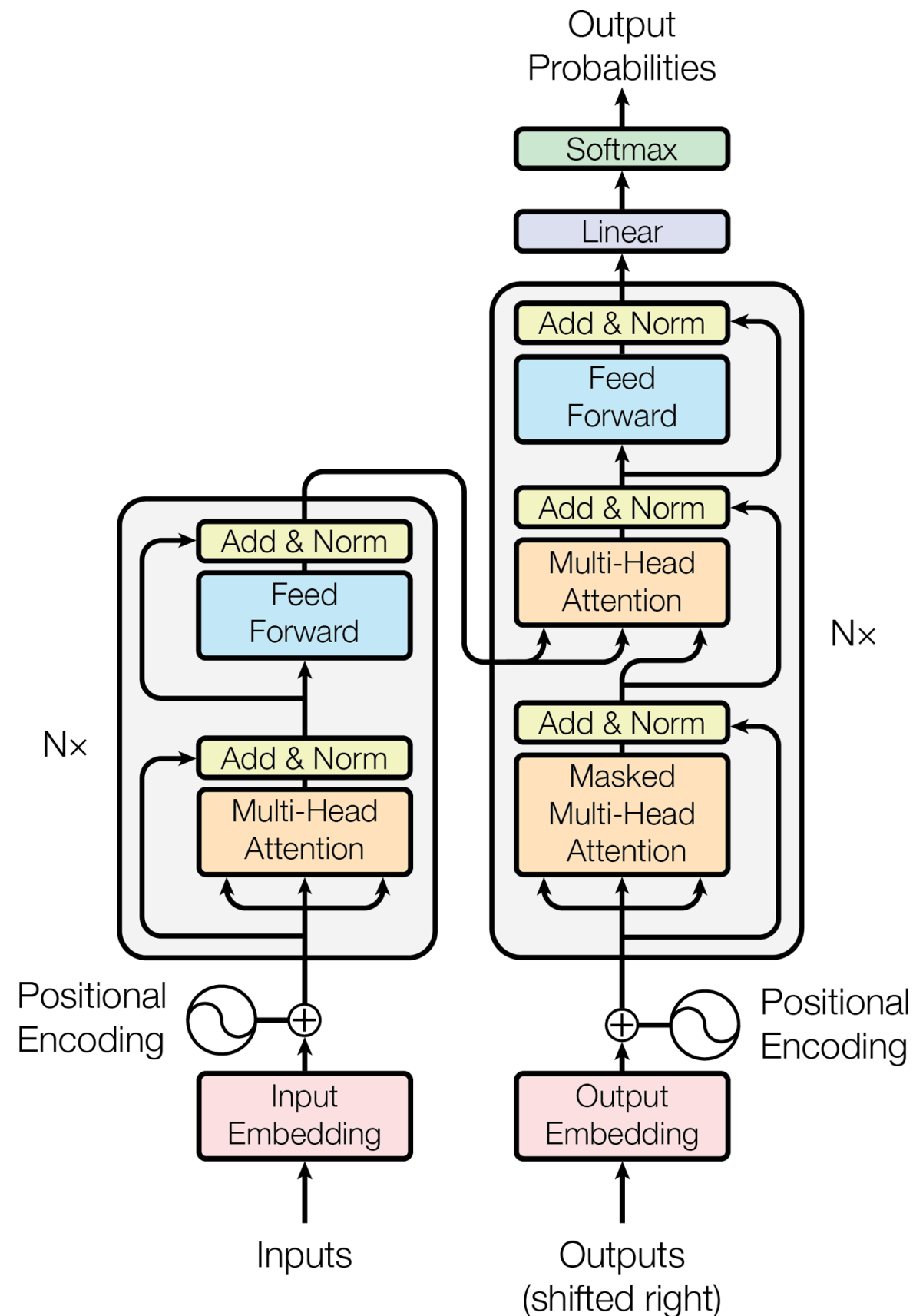
— From Recurrency to Attention

- Recurrent Neural Nets (RNNs) attempted to solve sequence modeling, but failed at scale



- Sequential processing: Impossible to parallelize (slow training)
- "Goldfish memory": Vanishing gradients cause the model to forget start of long reviews
- Transformers discard recurrency entirely in favor of **self-attention**
 - Parallelizable: Processes the whole sentence at once (much faster)
 - Long-range dependencies: Easily connects distant words (e.g., a "not" at the start to a "good" at the end)
- Seminal Paper: Vaswani, A., et al. (2017). *Attention is all you need*. In *Advances in neural information processing systems*.
 - Introduces the original Transformer architecture
 - The **baseline** for all modern LLMs (like BERT, GPT 1-4)

ARCHITECTURE

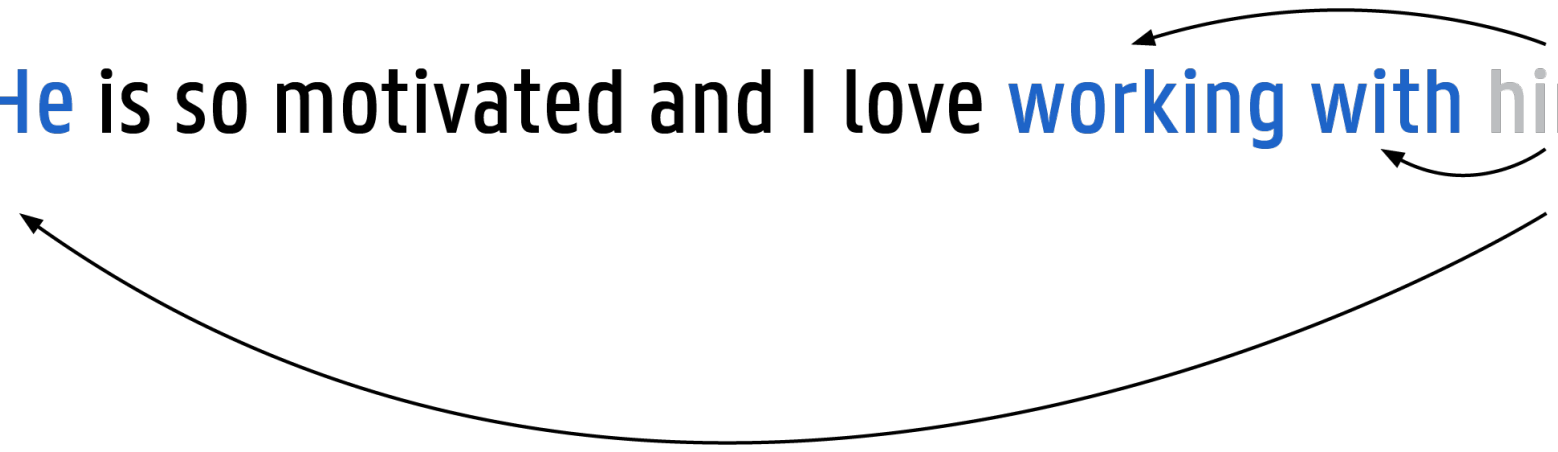


Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). *Attention is all you need*. In *Advances in neural information processing systems* (pp. 5998-6008).

ATTENTION

— Attention

John is my co-worker. **He** is so motivated and I love **working with** him



— Scaled dot-product attention

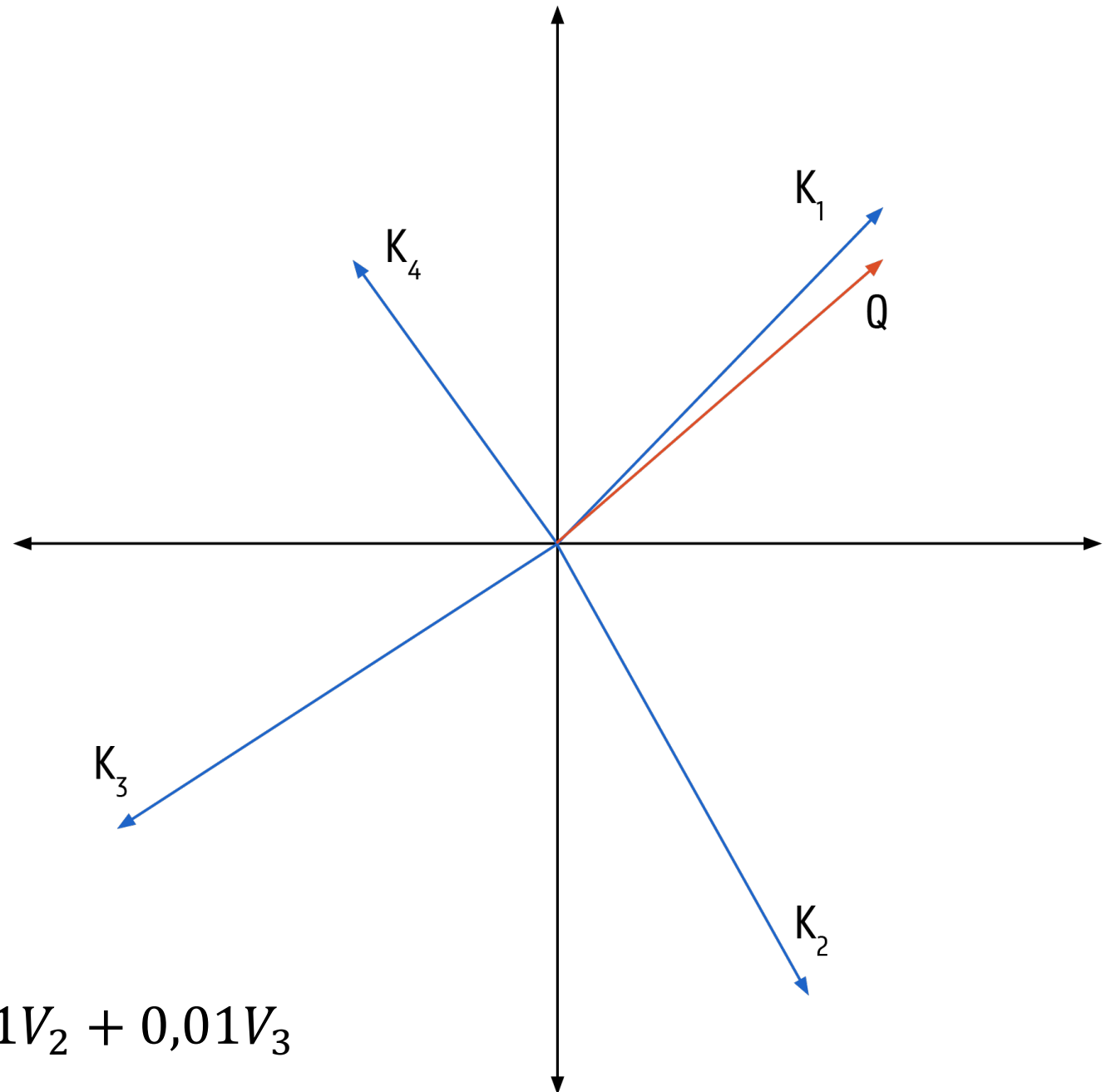
- $Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_{keys}}}\right) V$
- With Q a matrix of size $n_{queries} \times d_{keys}$, K a matrix of size $n_{keys} \times d_{keys}$, V a matrix of size $n_{keys} \times d_{values}$
- QK^T contains a similarity score for each query-key pair
- The final output has one row per query, where each row represents the query results

ATTENTION

Keys (K)	Values (V)
K_1	V_1
K_2	V_2
K_3	V_3
K_4	V_4

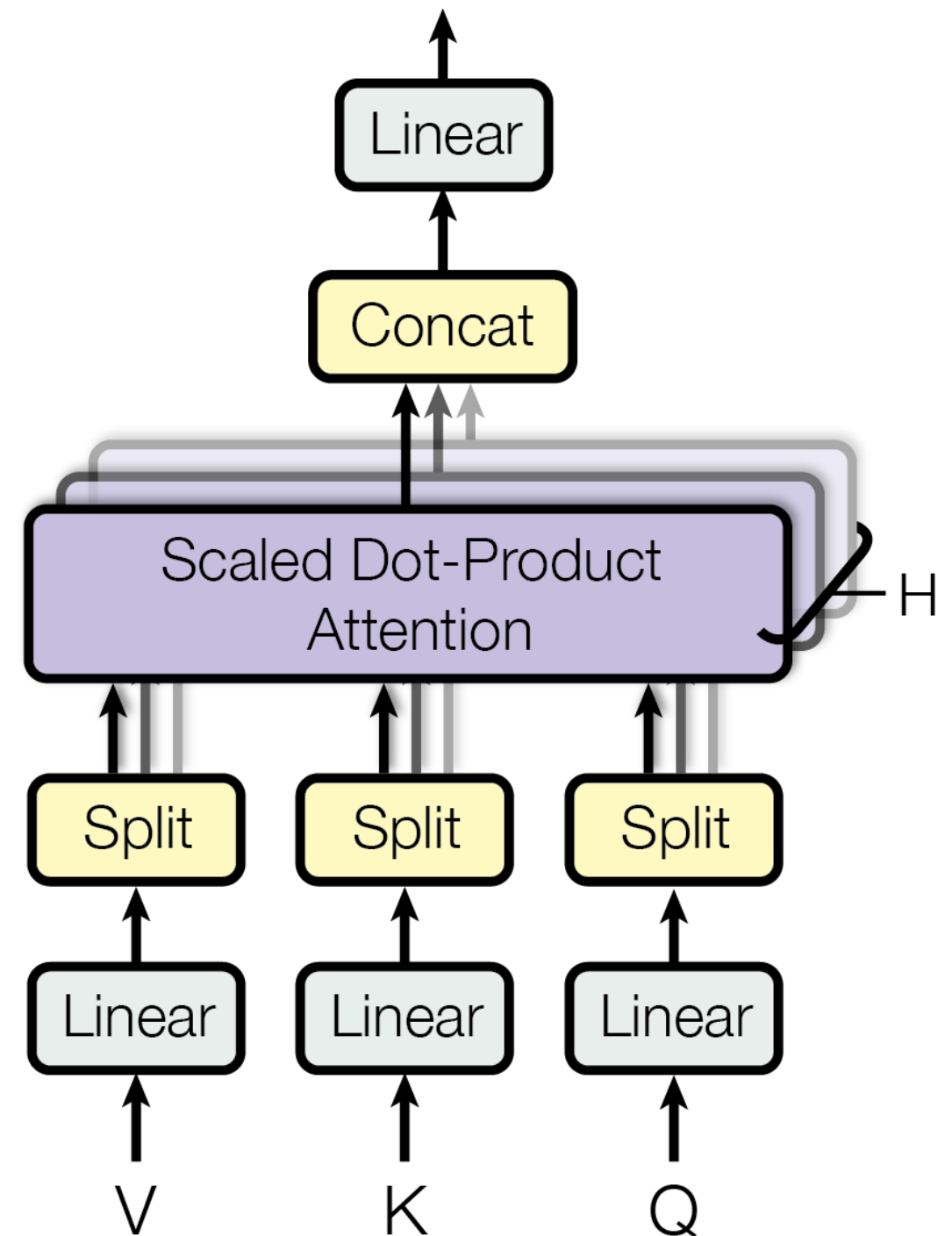
Inner product	Outcome
$\text{Softmax}(QK_1^T)$	0,98
$\text{Softmax}(QK_2^T)$	0,01
$\text{Softmax}(QK_3^T)$	0,01
$\text{Softmax}(QK_4^T)$	0,00

$$\text{Attention}(Q, K, V) = 0,98V_1 + 0,01V_2 + 0,01V_3$$



ATTENTION

- Multi-head attention
 - A number of **scaled dot-product attention layers** preceded by a **linear transformation**
 - A single attention layer would only keep **one characteristic** of the word
 - Multi-head attention applies multiple linear transformation and allows you to capture **different subsets of a word's characteristics**
 - E.g., 'he played a football game'
 - Single attention: played is a verb
 - Multi-head attention: played is a verb and past tense

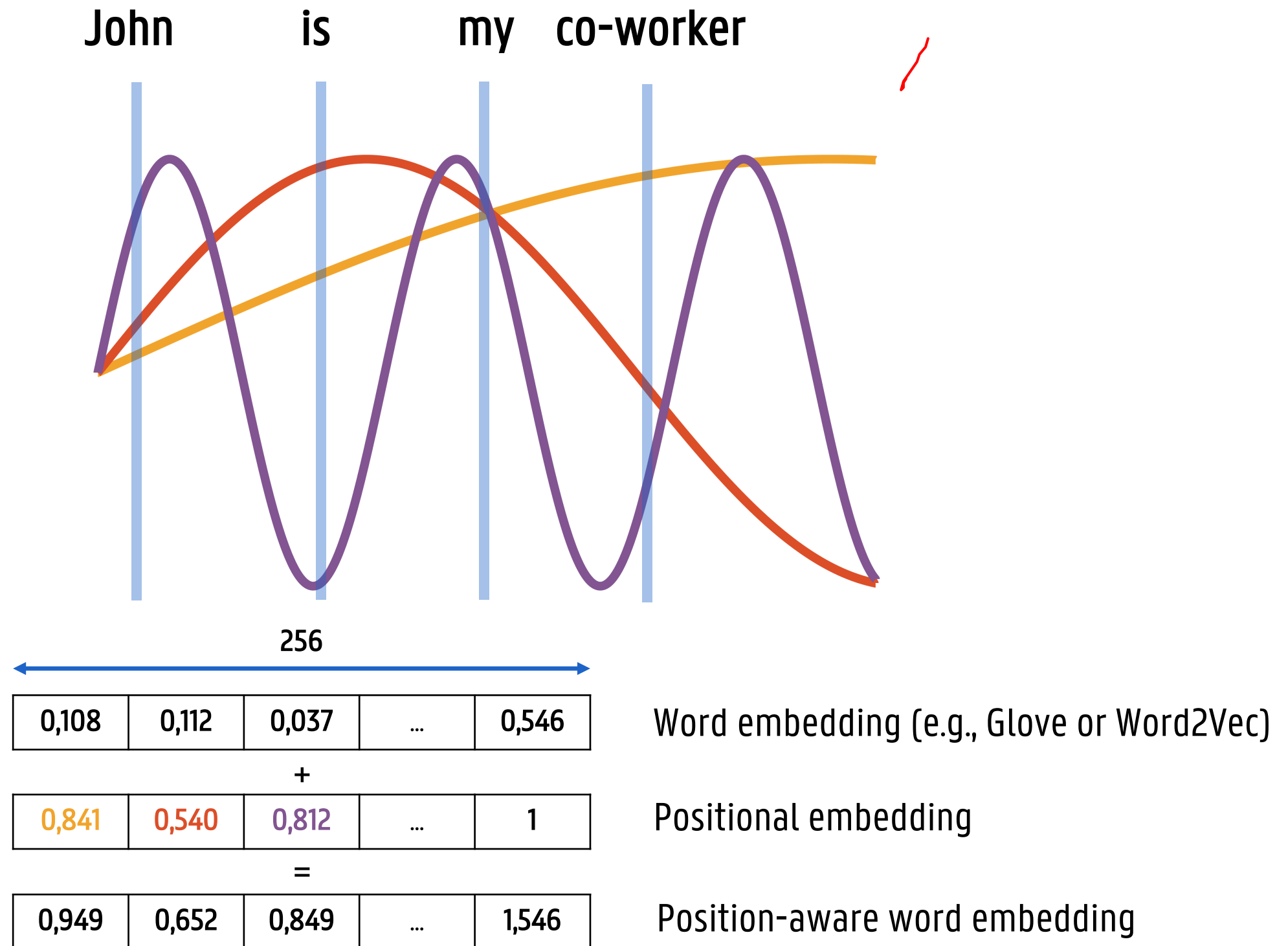


POSITIONAL ENCODINGS

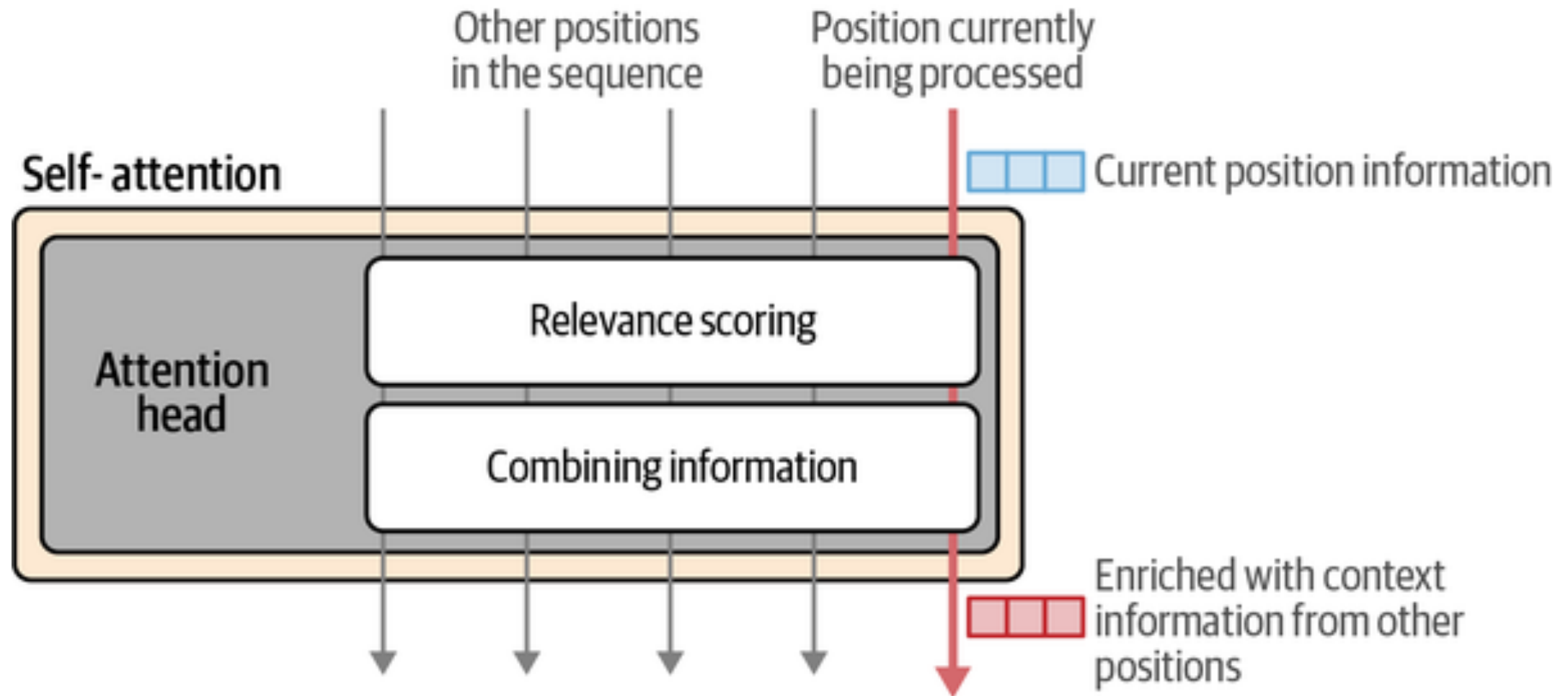
- Dense **vectors** that represent the **position** of the **word** in a **sentence**
- The i^{th} positional encoding is **added** to the word embedding of the i^{th} word in the sentence
- Since multi-head attention layers do not consider the order of the words, a positional encoding allows to give the **absolute** and **relative position** of the word
- Positional encodings can be **learned** but **fixed encodings** are preferred, with a sine and cosine function of different frequencies
- The **positional encoding** $PE_{p,i}$ of the i^{th} component of the encoding located at the p^{th} position is defined as:

$$PE_{p,i} = \begin{cases} \sin\left(\frac{p}{10000^{\frac{2i}{d}}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{p}{10000^{\frac{2i}{d}}}\right) & \text{if } i \text{ is uneven} \end{cases} \quad \text{with } d \text{ the embedding size}$$

POSITIONAL ENCODINGS



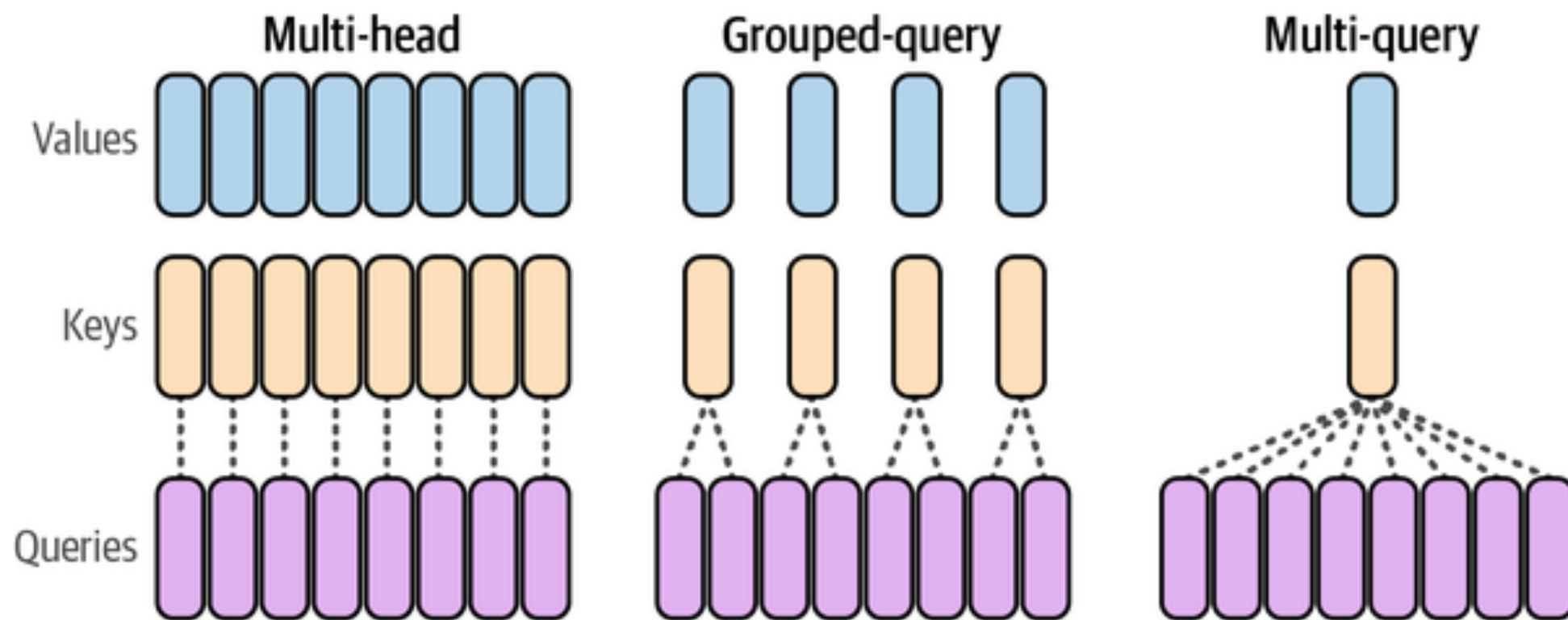
SUMMARY



- **Goal:** Generate a context-enriched positional embeddings for the current token
- **Relevance Scoring:** The current token's **Query** is multiplied against the **Keys** of all previous tokens to calculate an **Attention score** (how much focus to put on each word)
- **Weighted Aggregation:** These scores are multiplied with **Values** to produce the final output vector containing the relevant context

RECENT UPDATES TO THIS ARCHITECTURE

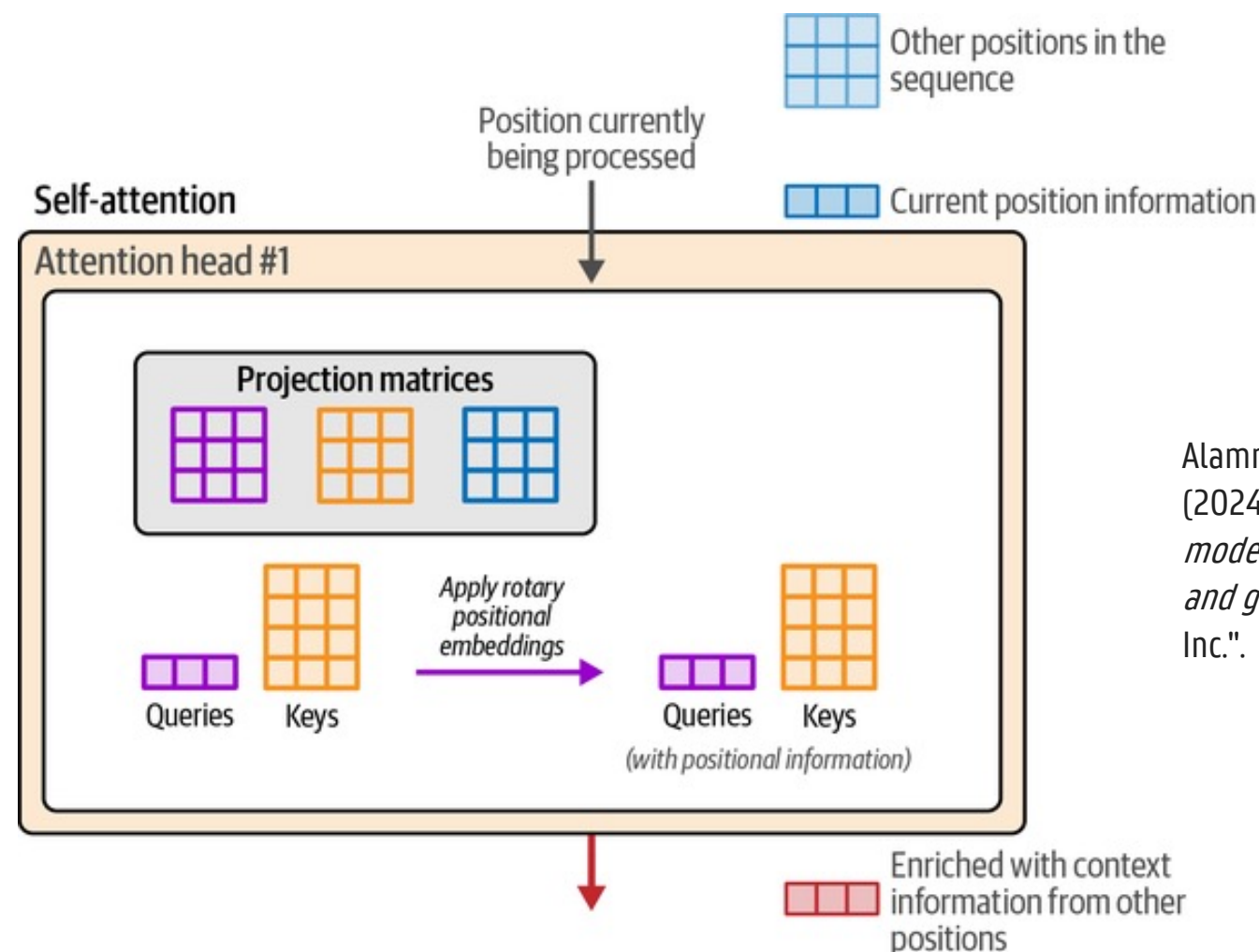
- **From multi-head attention to multi-query and group-query**
 - **Multi-head:** each attention head uses its distinct query, key and value matrices
 - **Multi-query:** The key and value matrices are shared accros all the attention heads but the queries are unique
 - **Grouped-query:** Group of attention heads share the same key and value matrices



RECENT UPDATES TO THIS ARCHITECTURE

— Rotary Positional Embeddings (RoPE)

- PE are **static** and **absolute** and added at the start of the forward pass of the transformer block which is inefficient for large context windows
- RoPE try to capture the **absolute** and **relative** token **position** by rotating the vectors in their embedding space in the attention step
- Rotation matrix is multiplied with the queries and keys just before computing the relevance score



Alammar, J., & Grootendorst, M. (2024). *Hands-on large language models: language understanding and generation.* O'Reilly Media, Inc."

TRANSFORMERS FOR SENTIMENT ANALYSIS

- If you want to use Transformer models for sentiment analysis, there are two options
- **Option 1: Encoder or Representation models (e.g., BERT, RoBERTa)**
 - The model looks at the entire sequence (**left and right**) simultaneously
 - It captures the **full context** and dependencies of every word relative to every other word
 - The primary **output** is a rich, high-dimensional embedding vector that encodes the meaning of the input text
- **Option 2: Decoder or Generative models (e.g., GPT-3, Llama)**
 - The model can only look at previous tokens (**left-to-right**)
 - It predicts the next token in the sequence based on the history (**auto-regressive attention**)
 - The primary **output** is a probability distribution for the next word, optimized for fluency and text completion

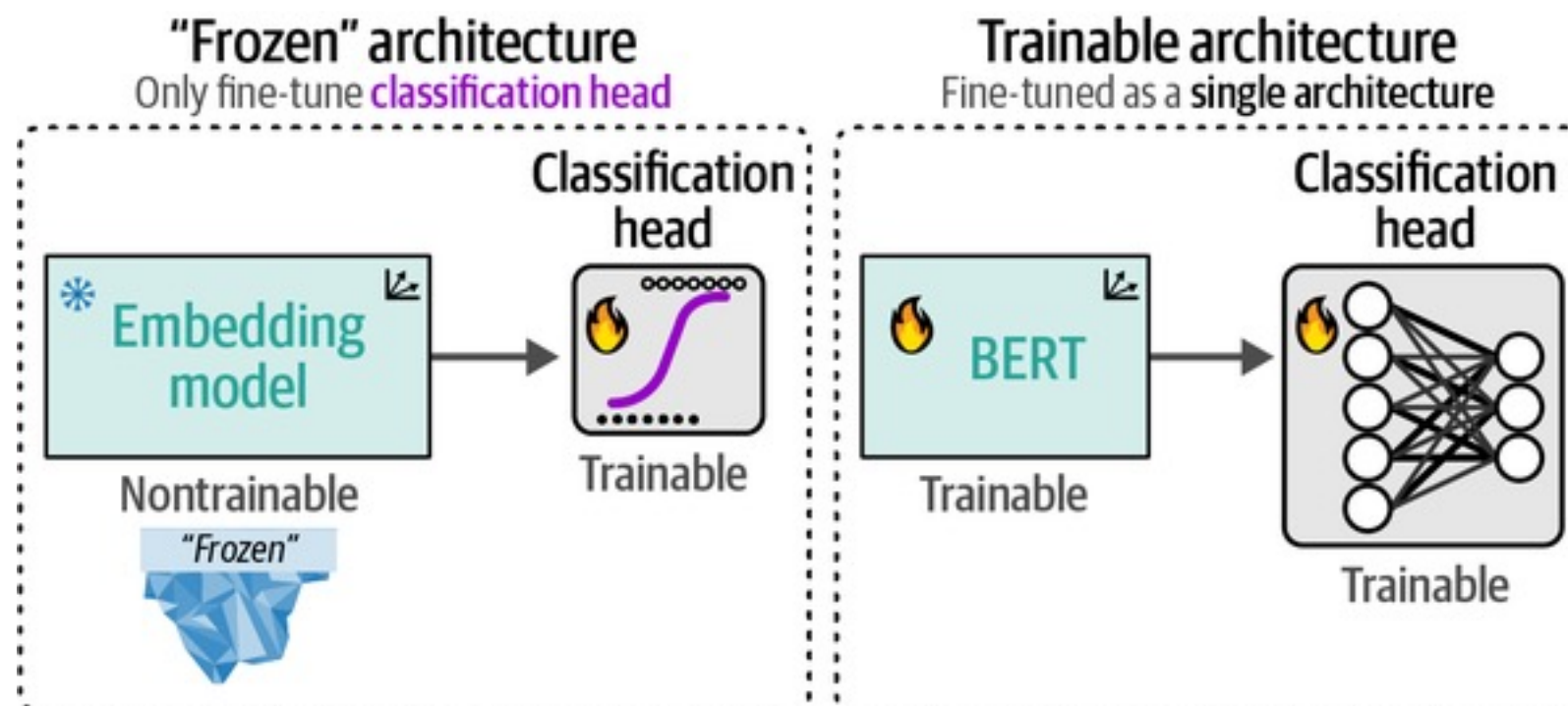
TRANSFORMERS FOR SENTIMENT ANALYSIS

- Based on these two options, there are also two strategies to compute sentiment.
- **Strategy 1: Fine-tuning (encoders)**
 - An encoder model is fine-tuned for a specific task (here, sentiment analysis)
 - This means that a pre-trained model is adapted to perform sentiment classification
 - The Encoder model's weights are either updated or used as features in a classification models
- **Strategy 2: Prompt Engineering (decoders)**
 - A structured **prompt** is provided to a Decoder model and asks the model to **generate** its **sentiment**
 - We do **in-context learning** and do not update the model weights

ENCODERS FOR SENTIMENT CLASSIFICATION

HOW?

- Sentiment classification is generally done by **fine-tuning** the model, which can be done in **three ways**:
 - **Task-specific model (transfer learning)**: use a pre-trained Encoder model that is already fine-tuned for a specific task and use it for inference.
 - **Fine-tuning the frozen architecture (featurization)**: the embeddings generated by pre-trained Encoder model are used as features in a downstream classification task
 - **Fine-tuning the (whole) architecture**: Both the weights of the pretrained Encoder model and the classification head are trained and updated jointly.



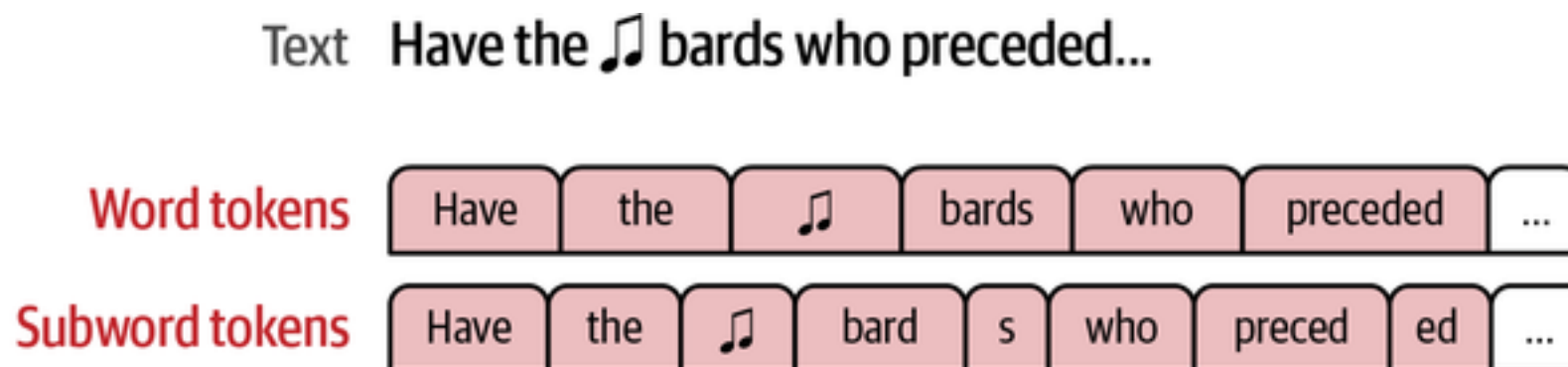
BERT

- **BERT = Bidirectional Encoder Representations from Transformers**
 - They outperform several state-of-the-art algorithms in various applications
 - *'BERT's state-of-the-art performance is based on two things. First, novel pre-training tasks called **bidirectional Masked Language Model (MLM)** and **Next Sentence Prediction (NSP)**. Second, a lot of data and compute power to train BERT'*
 - Trained on 3,3 billion records and on TPUs
 - Consists of two phases: pre-training and fine-tuning
 - <https://towardsdatascience.com/bert-technology-introduced-in-3-minutes-2c2f9968268c>



TOKENIZATION

- Tokenization is often the **core of a LLM** and can have substantial impact on its final performance
- Tokenization methods come up with an efficient set of tokens to represent text
- The **most popular** methods are: **byte-pair** encoding (GPT) and **WordPiece** (BERT)
- **WordPiece** is a **subword tokenization method**, meaning that it contains full and partial words
 - Compared to traditional word tokenization, subword methods break down words, it can easily handle new words



TOKENIZATION

- WordPiece has a **vocabulary size** of approx. 30,000 (uncased)
- **Special tokens**
 - [UNK]: An unknown token that the tokenizer has no specific encoding for
 - [SEP]: A separator that enables certain tasks that require giving the model two texts
 - [PAD]: A padding token used to pad unused positions in the model's input (as the model expects a certain length of input)
 - [CLS]: A special classification token for classification tasks
 - [MASK]: A masking token used to hide tokens during the training process
- Newline breaks are not considered
- The WordPiece tokenization will be used to break down the text and create the **token embeddings in BERT**

INPUT EMBEDDINGS IN BERT

Sentence: *"My dog is cute, he likes playing"*

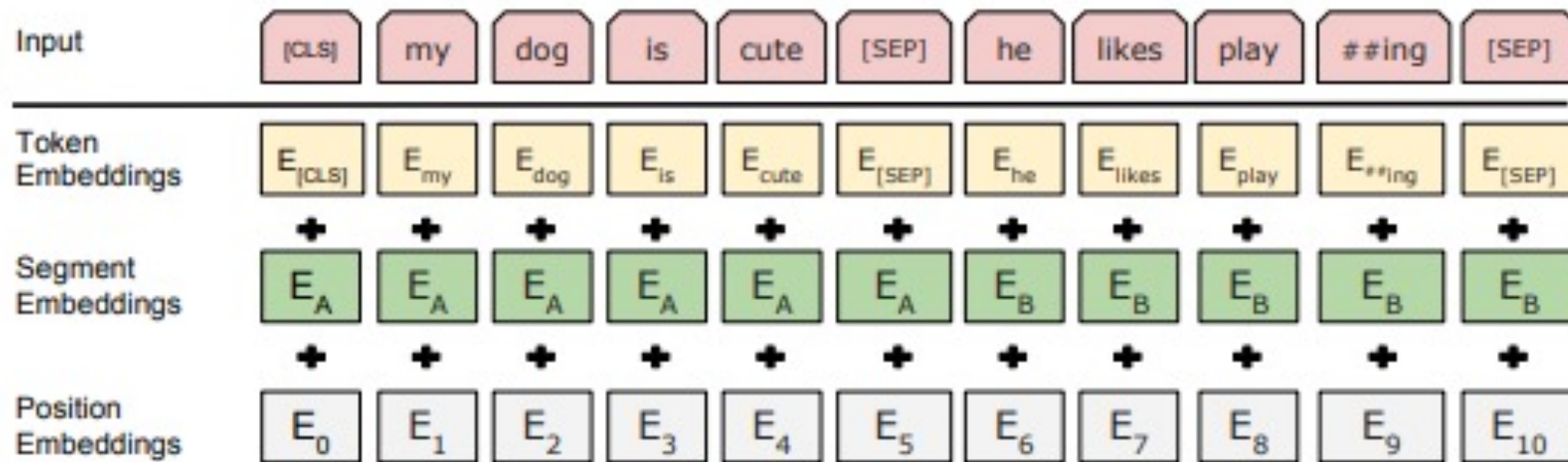


Figure 2: BERT input representation. The input embeddings is the sum of the token embeddings, the segmentation embeddings and the position embeddings.

Token embeddings are WordPiece embeddings with vocabulary size of 30,000

MASKED LANGUAGE MODELING

- Input sentence: my dog loses a lot of **hair** in the winter
- 15% of the words will be marked:
 - 80% of these words are masked: **[MASK]**
 - 10% of these words are replaced by a random word: **candy**
 - 10% of these words are kept: **hair**
 - The objective is to predict the correct identities of the marked words
- By applying this procedure, BERT is inherently **bidirectional** and **self-supervised**

NEXT SENTENCE PREDICTION

- A **balanced binary classification task** in which 50% is the correct next sentence (**IsNext**) and 50% not (**NotNext**)
- For example:
 - Input = [CLS] the man went to [MASK] store [SEP] he bought a gallon [MASK] milk [SEP]
 - Label = **IsNext**
 - Input = [CLS] the man [MASK] to the store [SEP] penguin [MASK] are flight ##less birds [SEP]
 - Label = **NotNext**

MOST POPULAR VARIANTS



— RoBERTa

- Robustly Optimized BERT removes next sentence prediction and increases the batch sizes and **training data** (10 times more up to 160gb)
- Consistently outperformed BERT (2 to 20% improvement) and often the standard nowadays

— DistilBERT

- Distilled BERT focusses on speed and efficiency and uses the concept of **Knowledge Distillation** in which a small student model mimics the teacher model.
- 40% times smaller and 60% times faster with minimal performance degradation (approx. 95% of BERT)

— ModernBERT

- Updates BERT with **modern LLM standards**, such as increased **training data** (from 3.3 billion to approx. 2 trillion tokens) and **context length** from 512 to maximum 8192 tokens.
- It also integrates some modern architectural concepts, like **rotary positional embeddings** and **flash attention**
- Ideal for tasks that require processing long documents and fit for multiple applications.

TRANSFER LEARNING OF A TASK-SPECIFIC MODEL



- There are over 6000 (!) pre-trained [sentiment analysis](#) models available on the Hugging Face Hub
- These models are **already fine-tuned on a specific dataset for sentiment analysis** using and can be used without retraining or labeling your model (zero shot learning)
- These models use the `sentiment-analysis pipeline` class, models can be built with 5 lines of code
- Popular models
 - [Twitter-roberta-base-sentiment](#) is a roBERTa model trained on ~124mio tweets and fine-tuned for sentiment analysis.
 - [Bert-base-multilingual-uncased-sentiment](#) is a model fine-tuned for sentiment analysis on product reviews in six languages: English, Dutch, German, French, Spanish and Italian
 - [Distilbert-base-uncased-emotion](#) is a model fine-tuned for detecting emotions in texts, including sadness, joy, love, anger, fear and surprise
- Specialized models also exist, such as [distilroberta-finetuned-financial-news-sentiment-analysis](#)

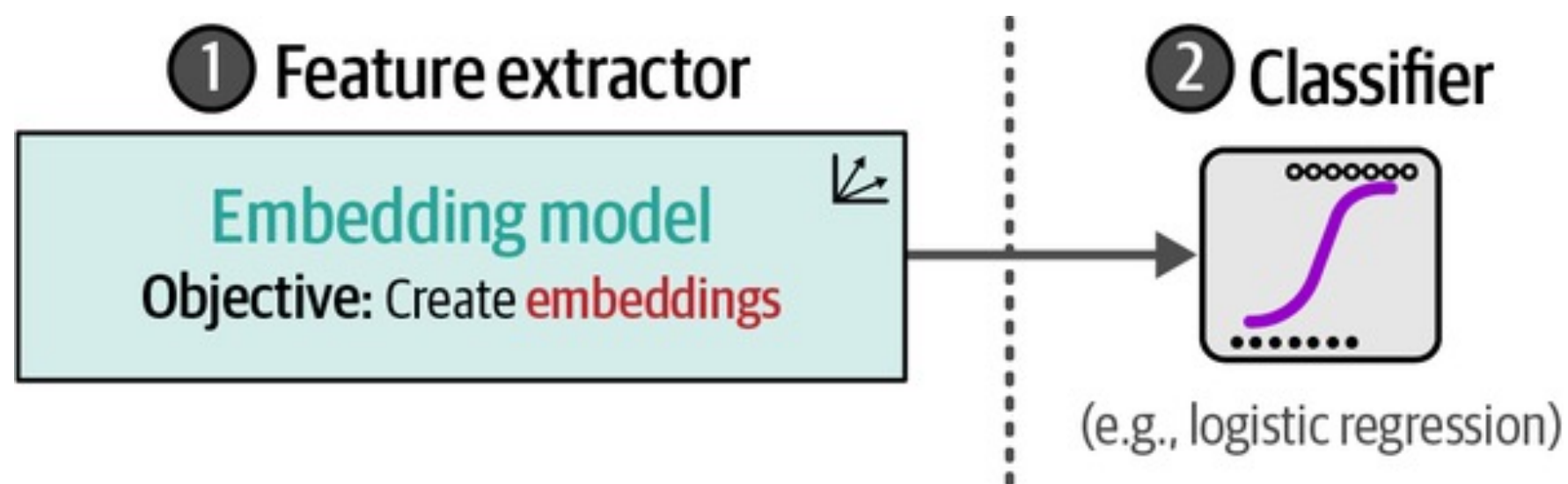
MOST POPULAR BERT MODELS FOR FINE-TUNING



- On **Hugging Face Hub** has over 100,000 models for [text classification](#) and more than 15,000 model for creating [embeddings](#)
- **BERT-models** are probably the most popular one for fine-tuning and embedding given their overall good performance, user-friendliness and small size (compared to GPT-models)
- Several alternative BERT-models are good starting points for finetuning :
 - [BERT base model \(uncased\)](#)
 - [RoBERTa base model](#)
 - [DistilBERT base model \(uncased\)](#)
 - [DeBERTa base model](#)
 - [Bert-tiny](#)
 - [ALBERT base v2](#)
 - [ModernBERT base](#)

FINE-TUNING WITH THE FROZEN ARCHITECTURE

- The most convenient way to fine-tune a BERT model is by using the pre-trained embeddings as features in a **two step process**:
 - The BERT model is used as a **feature extractor** to extract the **frozen** contextualized word embeddings
 - The embeddings (and potentially other) features are fed into your favorite **classifier**
- The main **benefits** are that it is **lightweight** and more **interpretable**
- The main **downside** is that the weights are not updated and can be less performant

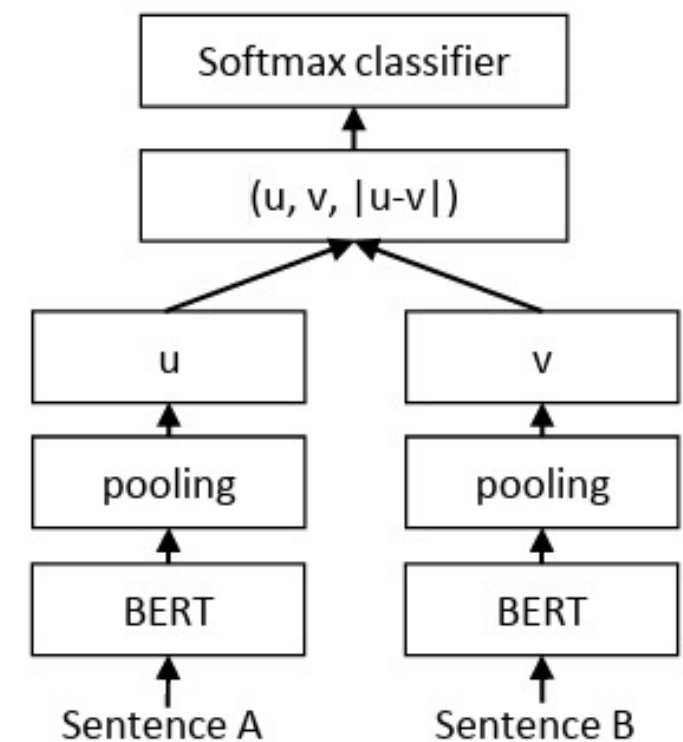


FINE-TUNING WITH THE FROZEN ARCHITECTURE

- **Problem: document-level embeddings**
 - All BERT-based models provide a contextualized embeddings for **every single token**
 - However, you need a single embedding that represent the **entire document** to feed into your sentiment classifier
- **Solution: Pooling strategies**
 - **[CLS] token**: take the vector of the special classification since this should serve as a 'global' summary
 - **Mean pooling**: take the average of all the word embeddings in a document
 - **Max pooling**: take the maximum value across each dimension
- All these strategies are suboptimal since the syntactic meaning is often destroyed
- **Better solution: Sentence-BERT**

SENTENCE-BERT

- Sentence-BERT comes up with an efficient embedding representation of a **sequence of text** (e.g., a sentence, document, or paragraph)
- In order to effectively train these sentence embeddings, Sentence-BERT uses a **Siamese architecture**:
 - **Two identical BERT models**, with the same weights and architecture are used
 - These models are fed with **two different sentences** for which embeddings are created using **pooling** (e.g., mean pooling)
 - Finally, the model is **optimized** through the **similarity** of the sentence embeddings



Reimers, N., & Gurevych, I. (2019). Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*.

SENTENCE-BERT

- The magic of Sentence-BERT is actually happening on how the model is **trained in terms of data and classification objective**
- **Data: the Stanford Natural Language Inference (SNLI) Corpus**
 - The SNLI corpus is a collection of 570k human-written English **sentence pairs** manually labeled for balanced classification with the labels: *contradiction*, *entailment*, and *neutral*
 - For example

Sentence A	Sentence B	Label
<i>A black race car starts up in front of a crowd</i>	<i>A man is driving down a lonely road</i>	Contradiction
<i>A soccer game with multiple males playing</i>	<i>Some men are playing a sport</i>	Entailment
<i>A smiling costumed woman is holding an umbrella</i>	<i>A happy woman in a fairy costume</i>	Neutral

SENTENCE-BERT

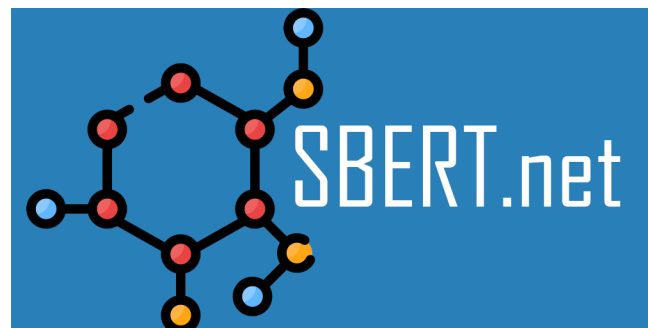
— Classification objective

- The **goal** of the final classification layer is to predict the label: entailment, neutral, or Contradiction
- Instead of feeding a classification head with only the embedding of sentence A (u) and sentence B (v), we **concatenate** 3 things ($u, v, |u - v|$)
 - This element-wise difference captures the distance between the two sentences, making it much easier for the final layer to decide if they are related.
- The final layer is a **Softmax linear transformation layer**: $\text{softmax}(u, v, |u - v|)$
- **Everything is trained together!**
 - During backpropagation, the gradients flow back from the weights of the Softmax layer through the weights of the two BERT models.
- **Final step:** after training, the final layer is deleted and the tuned BERT is to generate embeddings.

SENTENCE-BERT

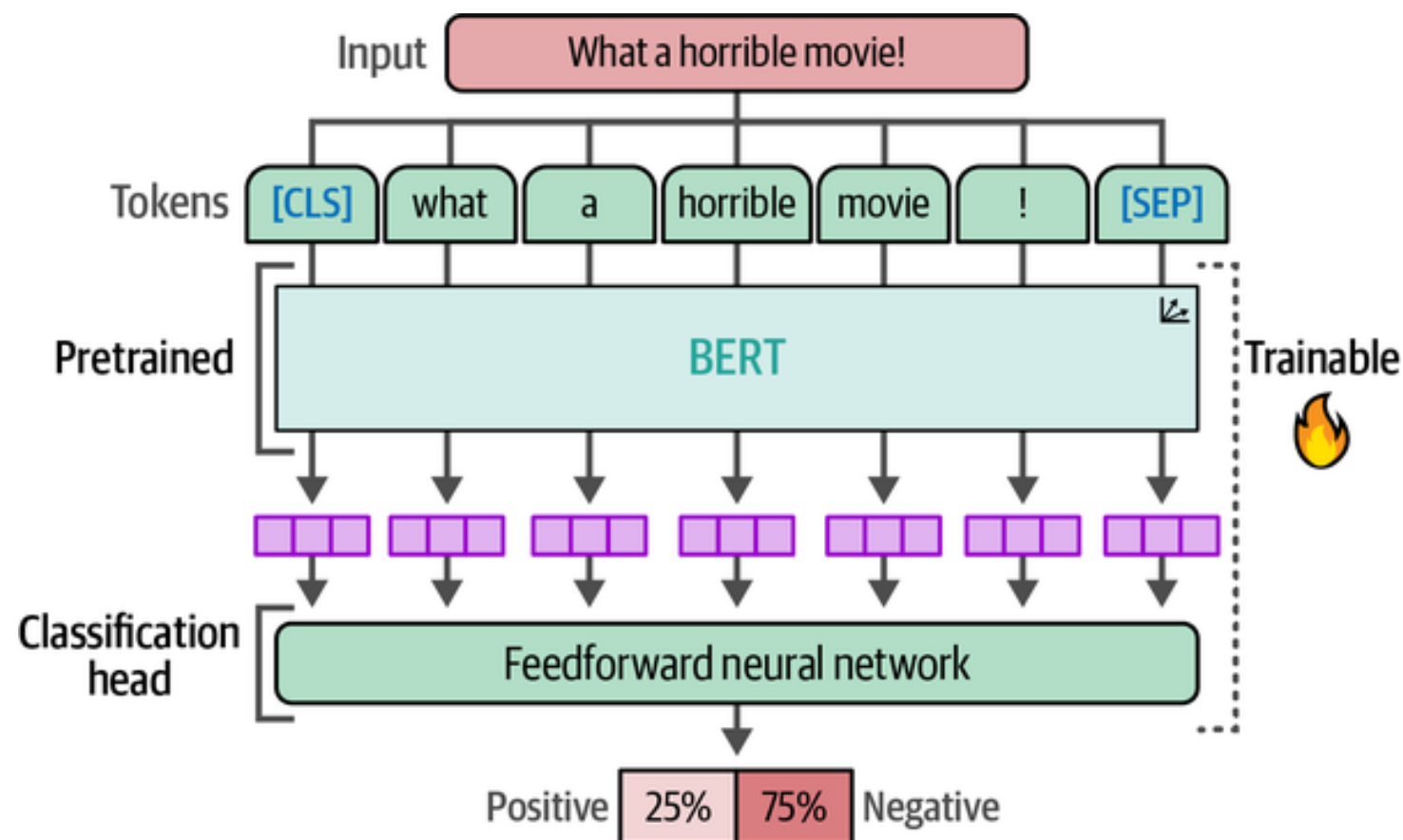
— Inference

- To generate embeddings the trained **BERT model** and the **pooling layer** are needed
- When you input a new sentence (e.g., *"This movie is sad"*).
 - **BERT model**: The model generates contextualized vectors for every token: ['This' , 'movie' , 'is' , 'sad']
 - **Pooling**: The Pooling layer mathematically averages these token vectors into **one single vector**
 - **Output**: You get one fixed-size vector (e.g., 768 dimensions) representing the whole sentence
- In this case average pooling does create **meaningful embeddings** since we specifically trained our model to do so!
- Note that this architecture can be theoretically applied to **any encoder model** and is easily implemented in Python with **sentence-transformers**



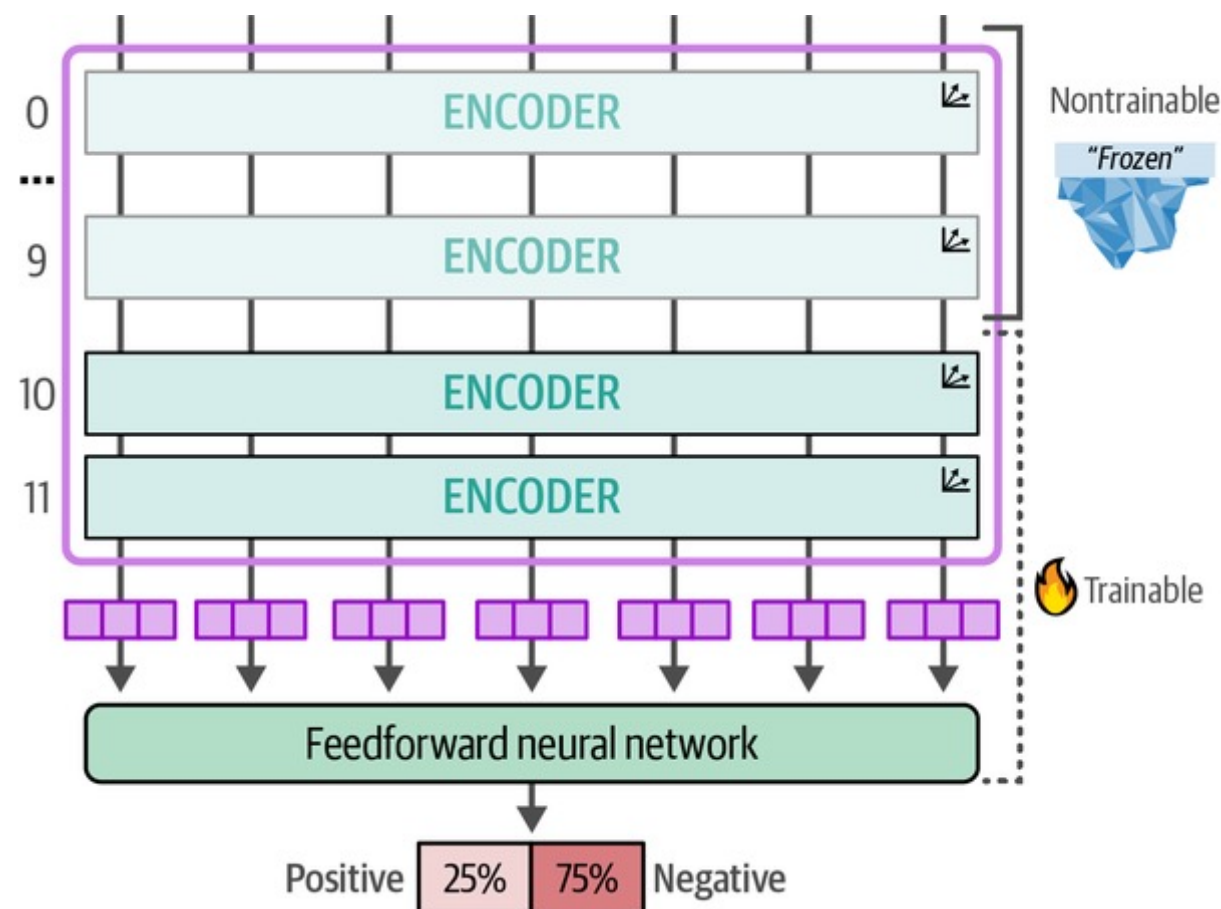
FINE-TUNING THE ARCHITECTURE

- In the featurization approach, the weights underlying BERT model remains frozen and only the classifier is trainable
- Another approach is to allow **both** the **BERT** model and the **classification head** to be **trainable**
- As such the BERT model and classification head are seen as a **single architecture** in which the parameters are updated or fine-tuned **jointly** during training.



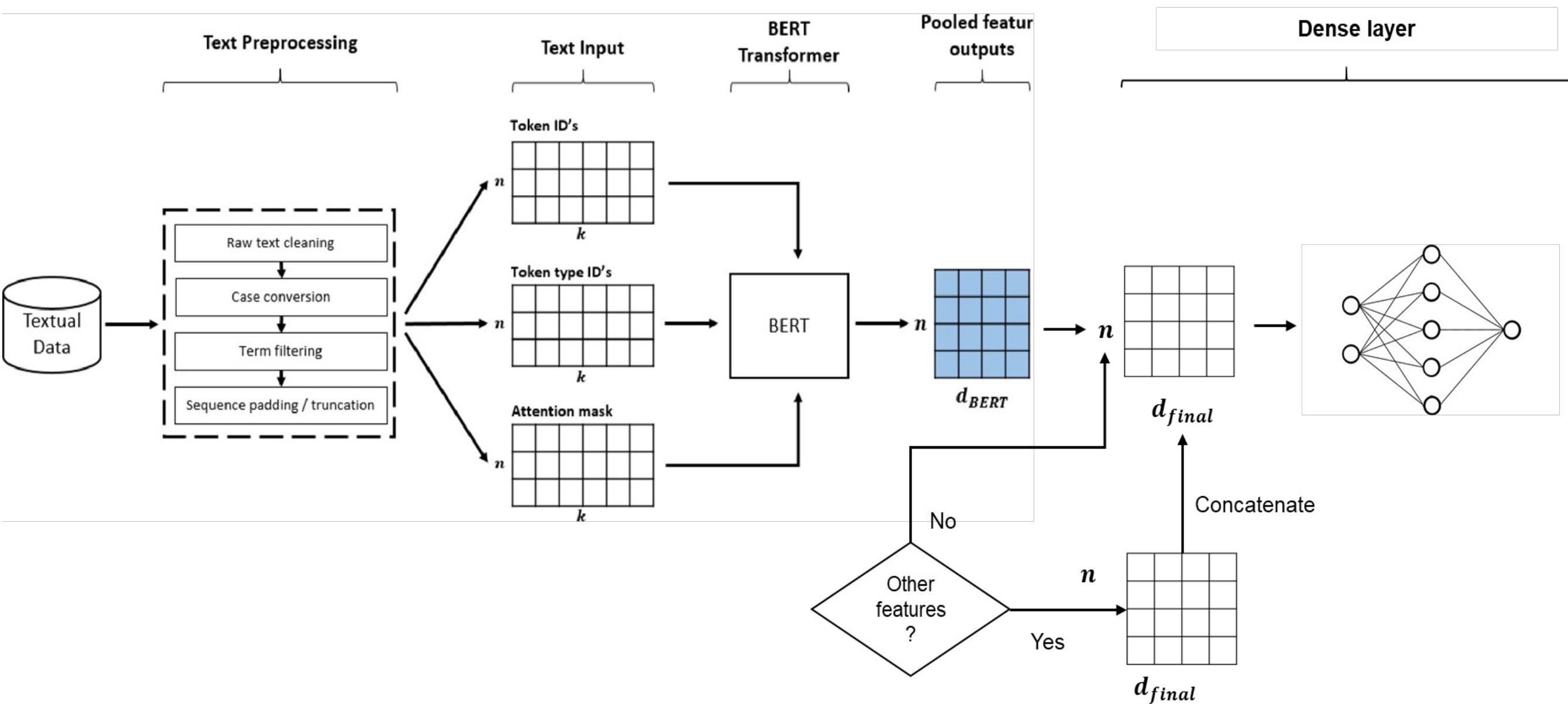
FINE-TUNING THE ARCHITECTURE

- Instead of updating all layers of your BERT model, you can also choose to only **freeze certain layers**
- The **idea** is that BERT learns in a hierarchy:
 - **Bottom layers**: Capture fundamental, universal features like syntax, grammar, and word parts
 - **Top layers**: Capture task-specific concepts like "sentiment," "logic," or "irony." These need to adapt to your specific task
- **The benefits**
 - **Computational overhead**: Fewer parameters to update means significantly faster computations
 - Prevents **catastrophic forgetting** on small datasets: By keeping the baseline solid, you stop the model from unlearning relations while trying to learn your specific task



SUMMARY OF FINE-TUNING WITH BERT

— Transformers/BERT set-up



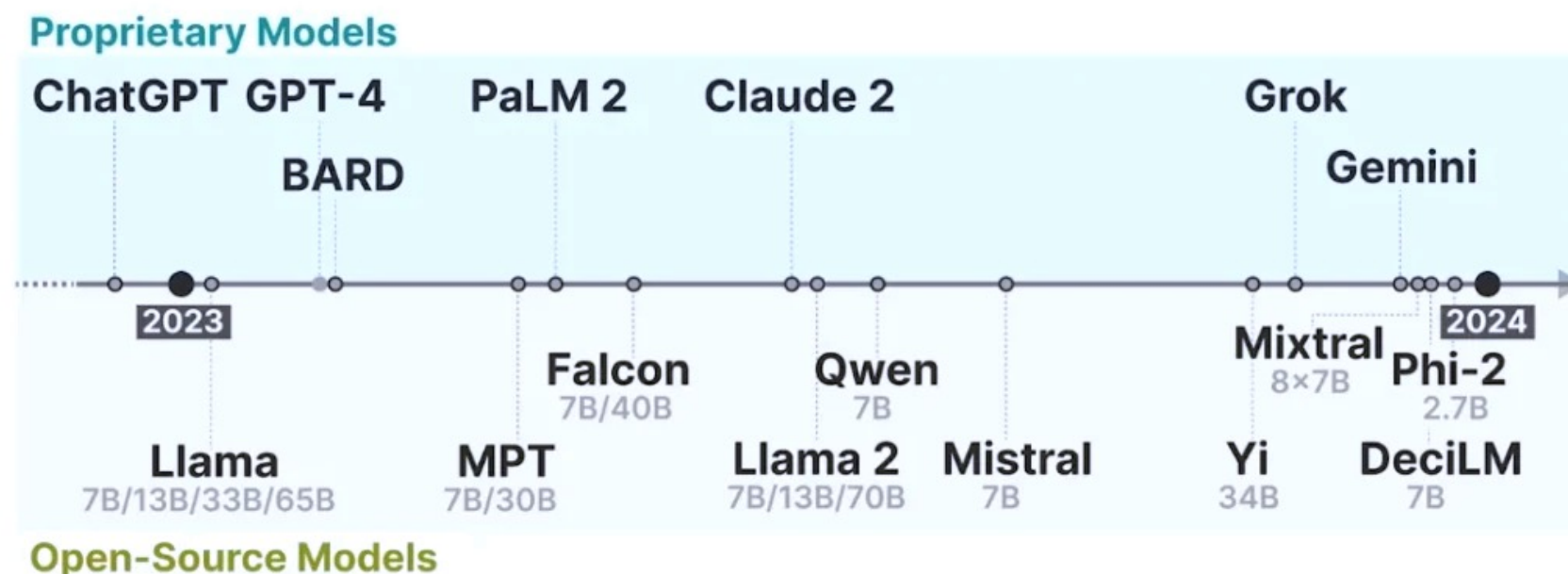
PROMPT ENGINEERING FOR SENTIMENT ANALYSIS

ENCODER VS DECODER MODELS

- **Encoder** models = output the sentiment class or probability based on an input sequence of tokens
- **Decoder** models = generate a sequence of tokens based on an input sequence of tokens
- **Problem:** Generative models are often general-purposes and not optimized to perform a specific task
- **Solution:** prompt engineering
 - Helping our LLM understand the context and guide it towards providing the right answers by means of carefully designed **prompts** or **instructions**

WHICH GENERATIVE MODEL TO CHOOSE?

- **Proprietary models** (e.g., GPT-4, Gemini, Claude)
 - Pros: State-of-the-art performance, fully managed (API)
 - Cons: Costly, "Black Box" (opaque), privacy risks, vendor lock-in
- **Open source models** (e.g., Llama, Qwen, DeepSeek, Mistral)
 - Pros: Free weights, data privacy control, flexibility (run locally or cloud)
 - Cons: Even small models require proper GPU hardware, trade-off size-performance
 - The gap is closing ! Models like Qwen and DeepSeek are quite competitive with proprietary models
- There is a huge **variety** in open source LLMs based on the training data and the number of parameter sizes (from <1B to 70B+ parameters)
- **Challenge:** Which one from these thousands of to select?

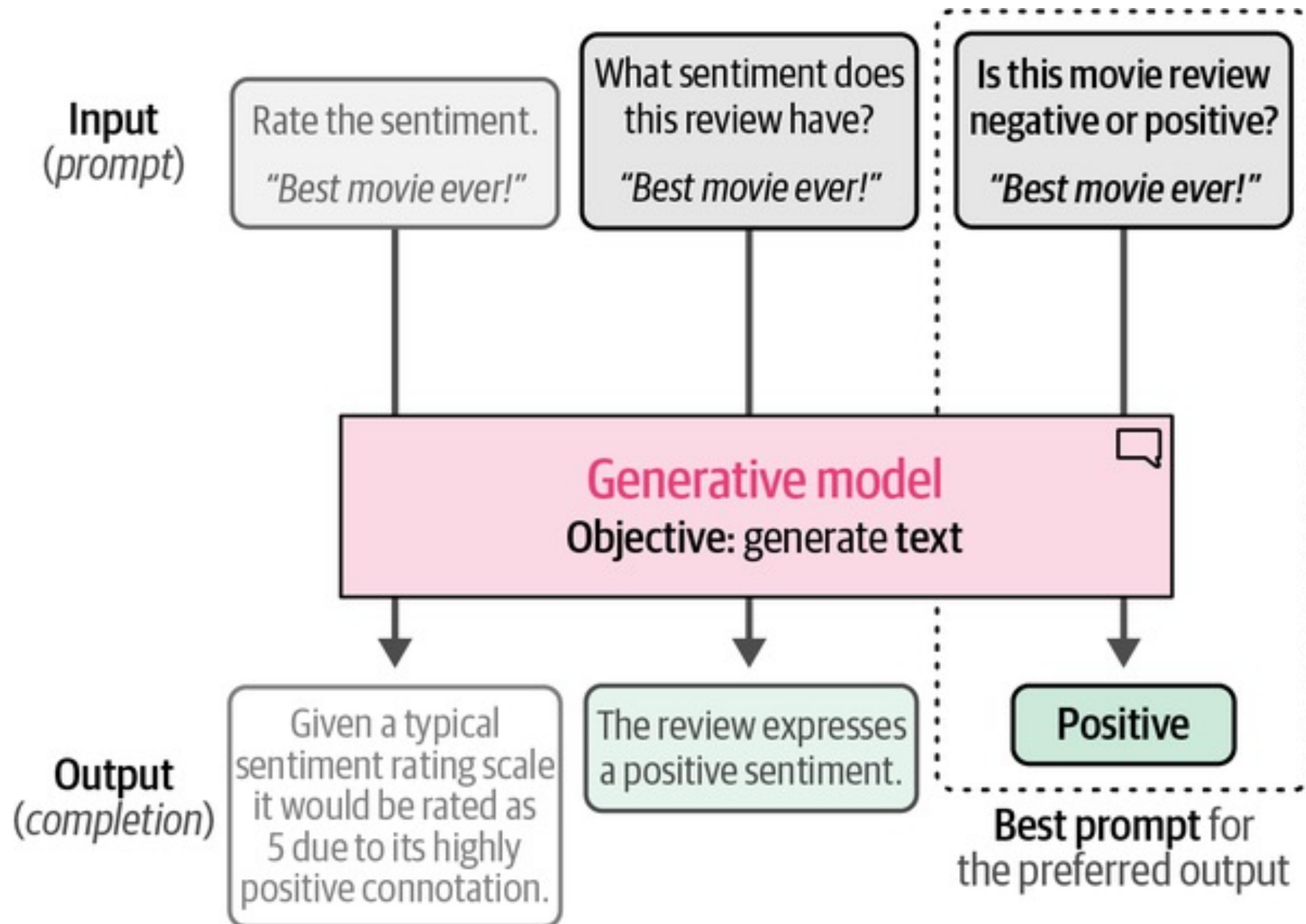


WHICH GENERATIVE MODEL TO CHOOSE?



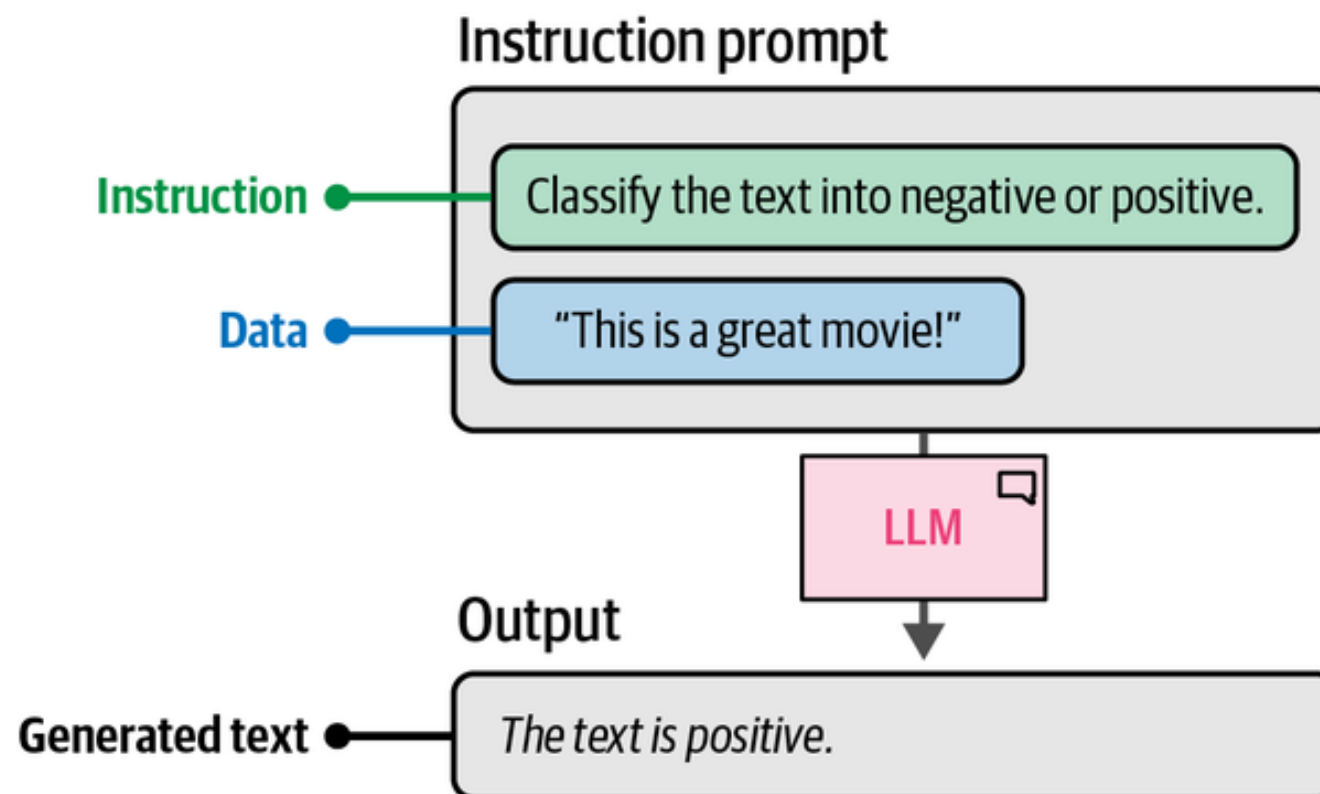
- **The most “popular” small open source models**
 - **Llama 3 (Meta):** The industry standard. Excellent for general chat and widely supported, but the 8B version pushes the limits of standard laptops (8GB VRAM).
 - **Qwen 2.5 (Alibaba):** A performance powerhouse, especially in coding and logic, often outperforming larger models.
 - **Phi-3 (Microsoft):** Designed specifically for "efficiency." It offers high reasoning capabilities in a tiny package (3.8B parameters).
- **Our choice: Phi-3-mini (3.8B parameters)**
 - **Hardware friendly:** Fits comfortably in **8GB VRAM** (household GPUs).
 - **Fit to learning prompt engineering:** Excellent to show impact of prompts and formatting
 - **Transparency:** We can run it locally to see exactly how parameters (e.g., temperature, top-p) affect the raw output and don't need API access
- Since we use the Hugging Face Hub, **switching** to Qwen or LLaMA is straightforward!

PROMPTING FOR SENTIMENT ANALYSIS



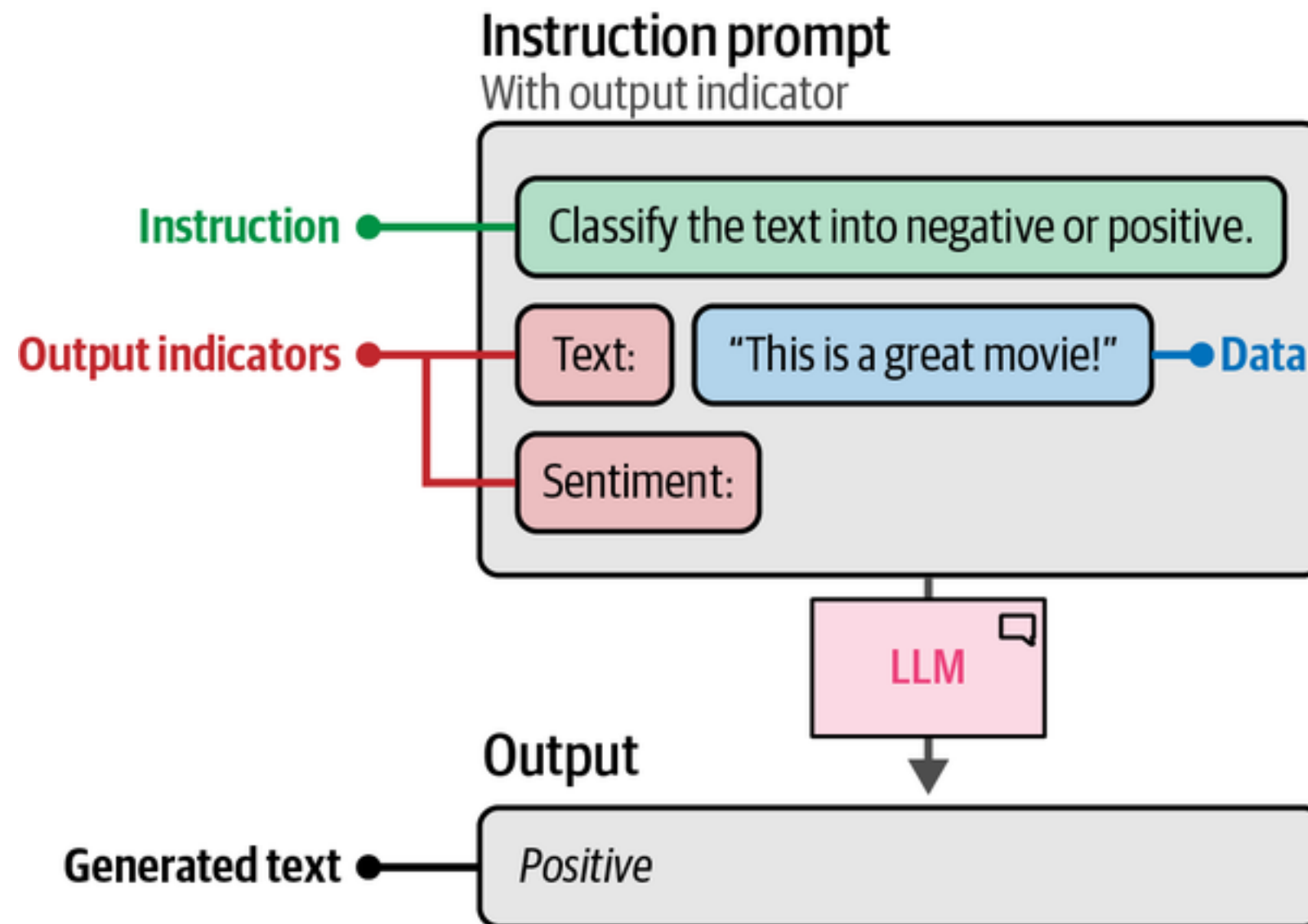
INTRODUCTION TO PROMPT ENGINEERING

- The performance of a generative model will depend on a proper design of your prompt and often requires optimization and experimentation
- However, there are some **guidelines** that you can follow in order to make your prompt more structured and fit to your task
- **Basic instruction-based prompting:** contains a simple instruction and the data



INTRODUCTION TO PROMPT ENGINEERING

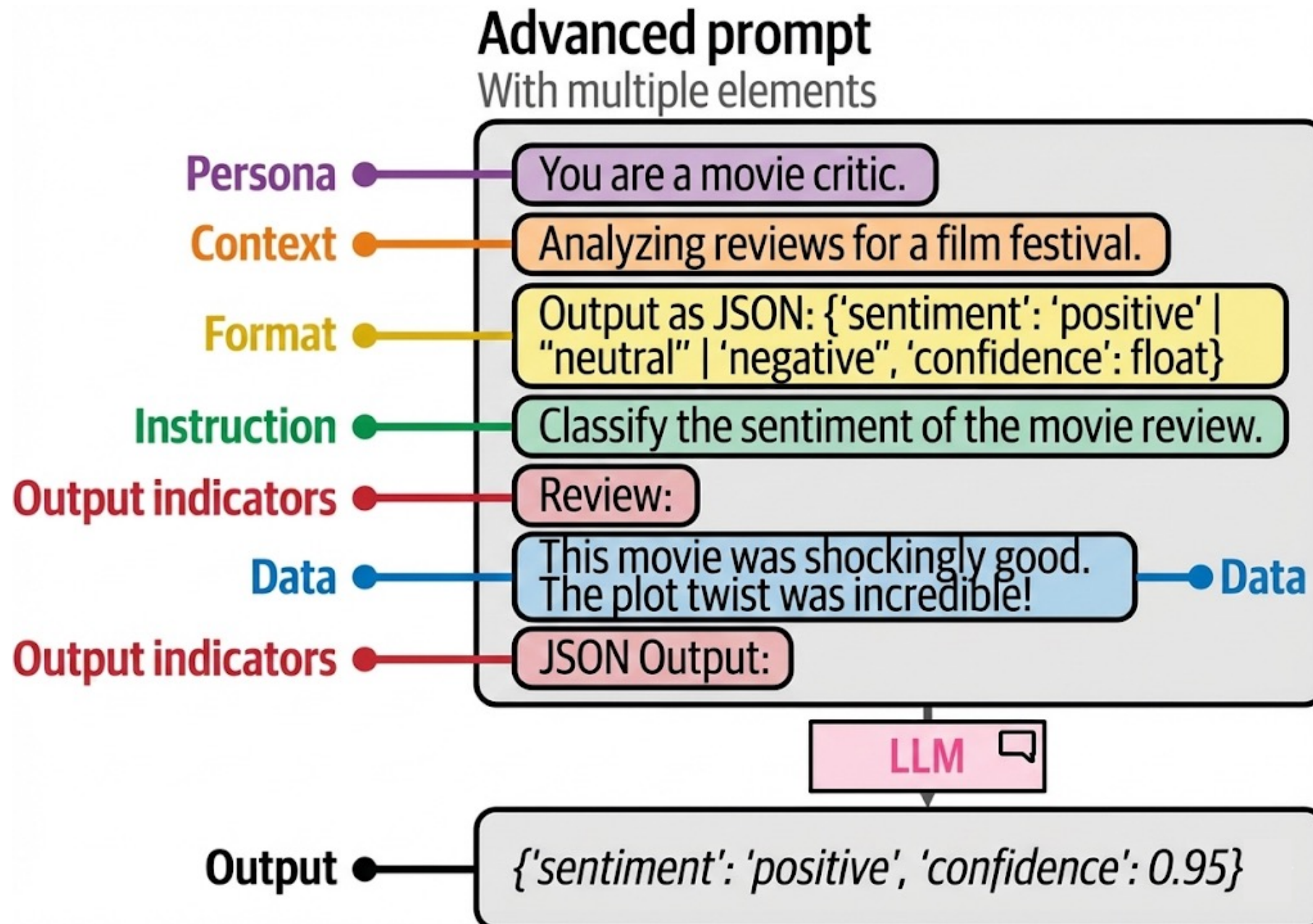
- **Complex instruction-based prompting:** next to the instruction also specify the desired output (format)



MORE ADVANCED PROMPT ENGINEERING

- Besides the 3 **basic ingredients** (instruction, data, and output) more **advanced elements** can be added to enrich your prompt:
 - **Persona**: Describing the role that the LLM should be
 - **Instruction**: A specific instruction of the task
 - **Context**: Additional information about the context of the task that matters
 - **Format**: How should the LLM output the generated text. For example, {"sentiment": "positive" | "neutral" | "negative", "confidence": float}.
 - **Audience**: The target audience for the generated output text
 - **Tone**: Should it be formal, informal?
 - **Data**: The main data on which the instruction should be performed
 - **Output**: Specify the desired output
- In sentiment analysis, the additional elements that could improve performance are the **output, context and persona**.
- It still remains **trial-and-error** to see which elements should be added or not and in which order!
- Since sentiment analysis is a quite straightforward task, **don't over-engineer!**

ADVANCED PROMPT ENGINEERING



IN-CONTEXT LEARNING

- Instead of adding additional elements to the prompt that describe what it should do, a better option could be to show some examples!
- **Few shot prompting:** provide the models with concrete examples to improve task-specific performance

Zero-shot prompt

Prompting without examples

Classify the text into neutral, negative, or positive.

Text: I think the food was okay.

Sentiment: ...

One-shot prompt

Prompting with a single example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.

Sentiment: **Neutral**

Text: I think the food was okay.

Sentiment:

Few-shot prompt

Prompting with more than one example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.

Sentiment: **Neutral**.

Text: I think the food was great!

Sentiment: **Positive**.

Text: I think the food was horrible...

Sentiment: **Negative**.

Text: I think the food was okay.

Sentiment:

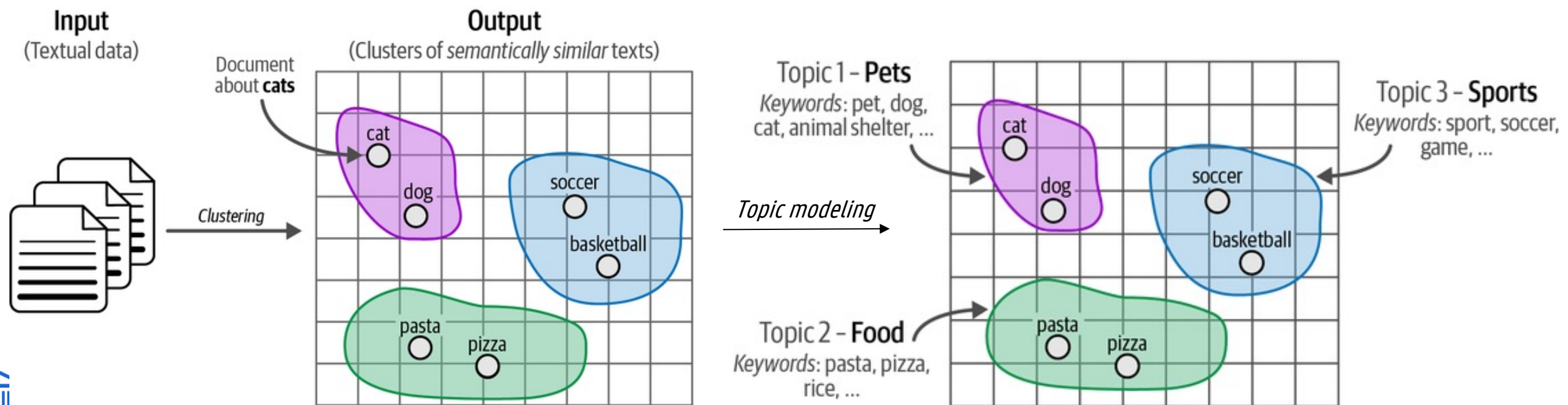
EVEN MORE ADVANCED ENGINEERING?

- Sentiment analysis is primarily a **classification task**, not a complex riddle
- **Simple, structured prompts** are usually sufficient and more **efficient**.
- Some additional techniques are an **overkill** in this case:
 - **Chain Prompting**: Unnecessary for single-step classification, the task is direct and doesn't require breaking down into sequential sub-tasks
 - **Chain-of-Thought**: Asking for step-by-step reasoning is overkill for simple labeling and can sometimes lead to over-interpretation
 - **Self-Consistency**: Since chain-of-thought is already excessive, generating and voting on multiple paths is highly inefficient and begs for overfitting
 - **Tree-of-Thought**: Designed for strategic problem-solving and backtracking, not for direct text-to-label mapping

BERTOPIC

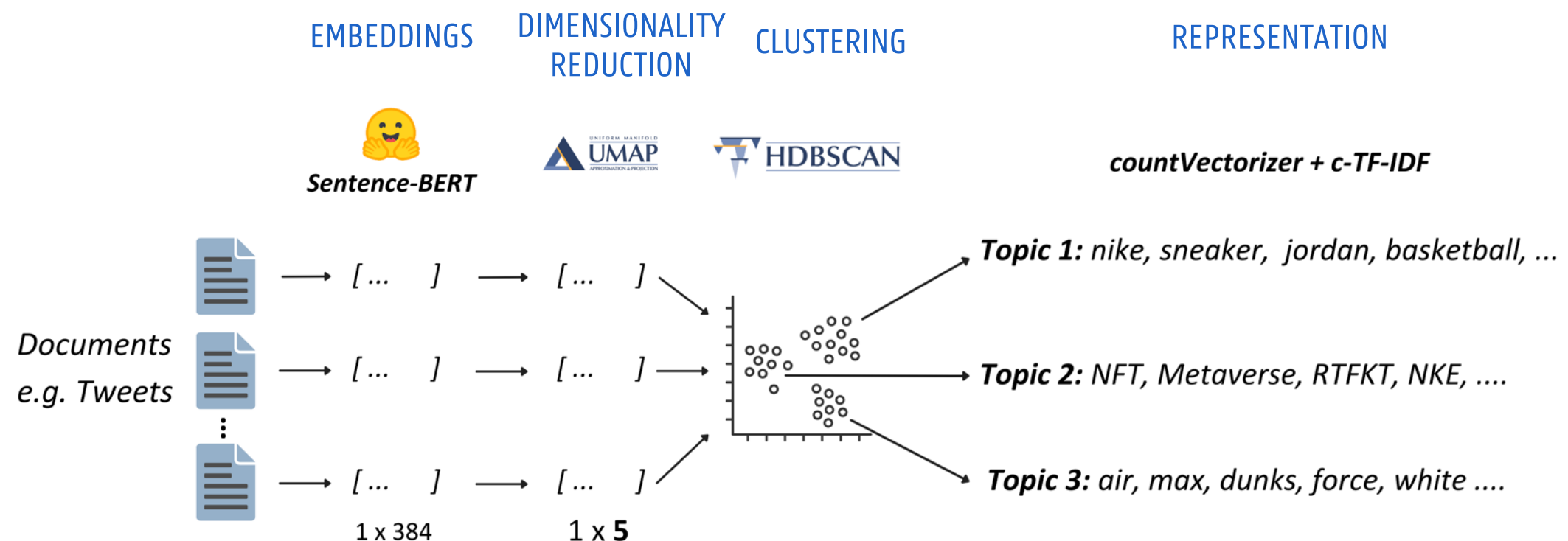
TOPIC CLUSTERING

- **Remember:** topic modeling tries to find the underlying latent topics in a collection of documents
- **Text clustering:** cluster similarity documents based on their semantic content, meaning or relationship
- Topic modeling can be seen as a way to provide **meaning** to the clustering of document
- **BERTopic** can be seen as text-clustering-inspired way of performing topic modeling



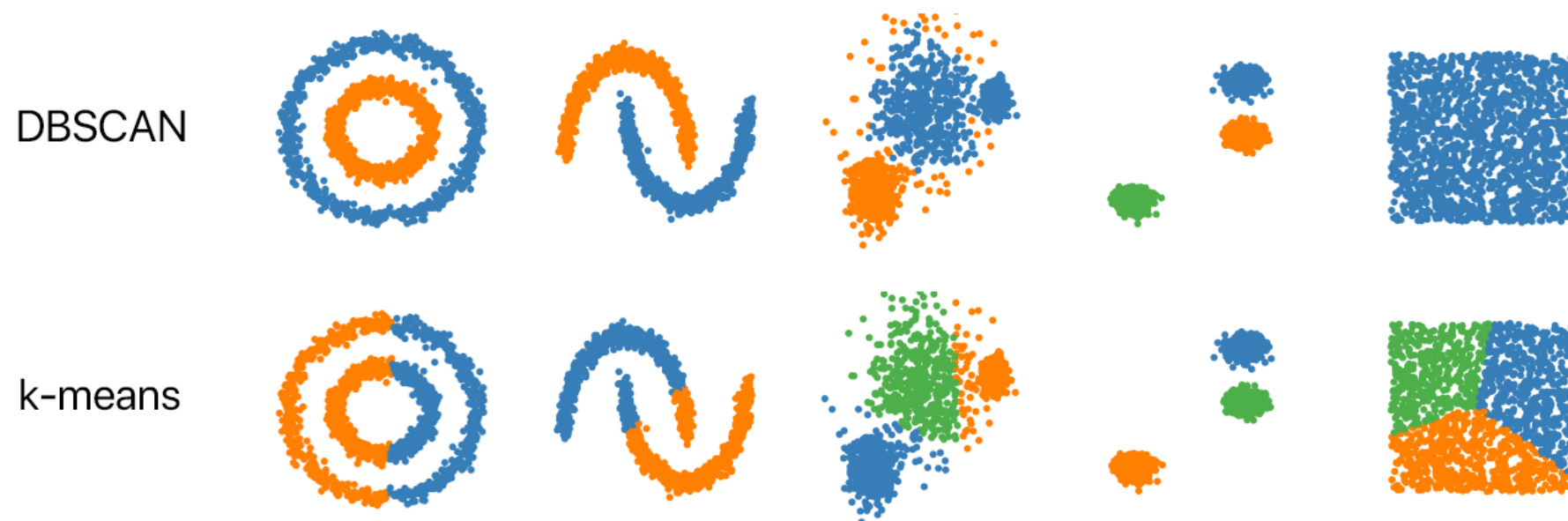
BERTOPIC

- BERTopic is a **neural embedding approach** to topic modeling that leverages clusters of semantic similar documents to extract various types of topic representations
- It follows a **modular procedure** based on **text clustering**: **embedding** documents, **reducing** their dimensionality, **clustering** these reduced embeddings to create semantic similar clusters and finally **representing** these clusters as topics



HDBSCAN

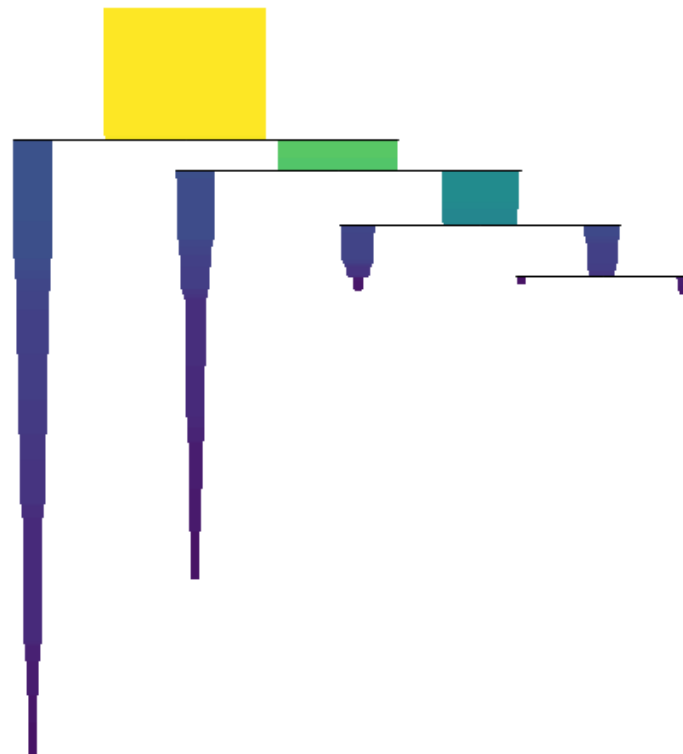
- **HDBSCAN** = Hierarchical Density-Based Spatial Clustering of Applications with Noise
- A **hierarchical version** of **DBSCAN** that allows for more dense clusters to be found
- Both methods are **density-based methods**: observations are clustered together in high-density regions and others are seen as outliers
- They don't require the number of topics to be specified upfront



HDBSCAN

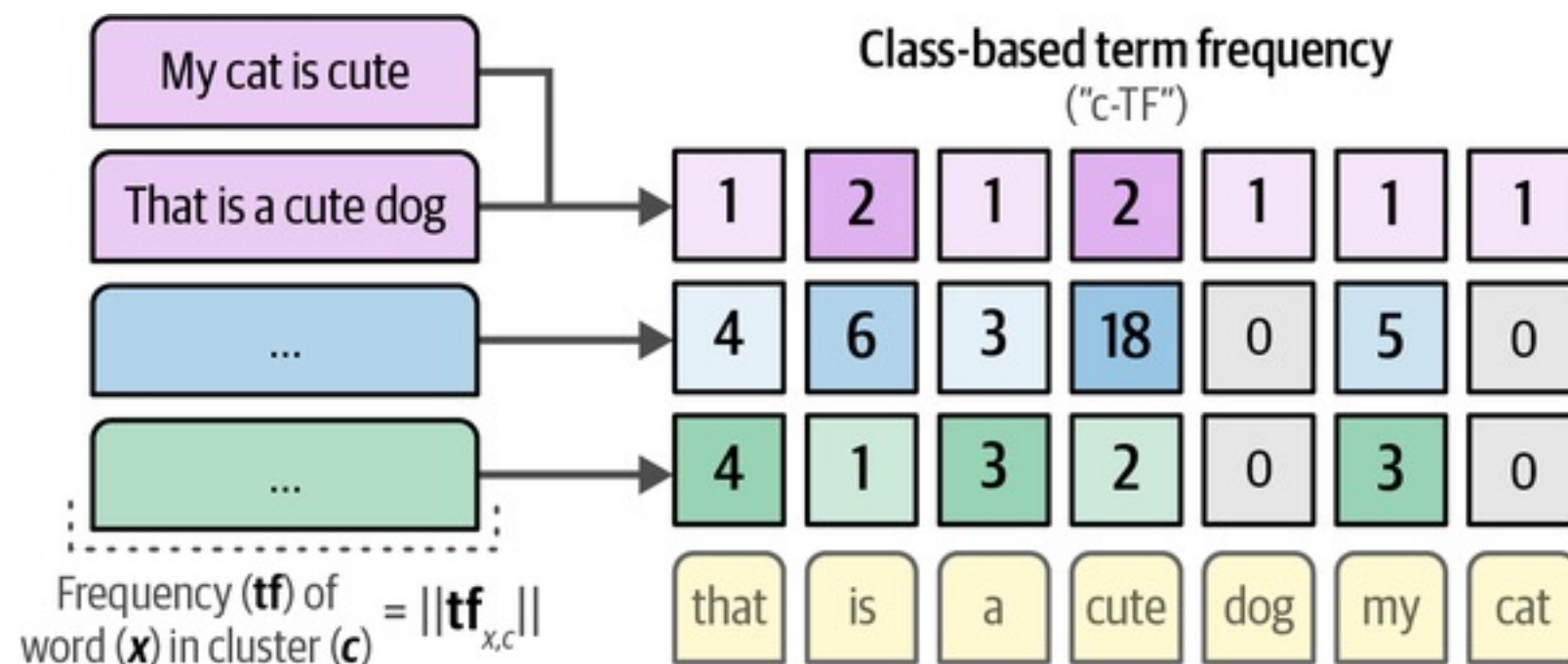
Feature	DBSCAN	HDBSCAN
Density Assumption	Uniform (same density everywhere)	Varying (dense & sparse mixed)
Method	Fixed Radius (ϵ)	Hierarchical tree (or dendrogram)
Epsilon (ϵ)	Global (One value fits all)	Adaptive and Locally adjusted
Best For	Simple, consistent geometric data	Complex, real-world data (e.g., text)

— Example



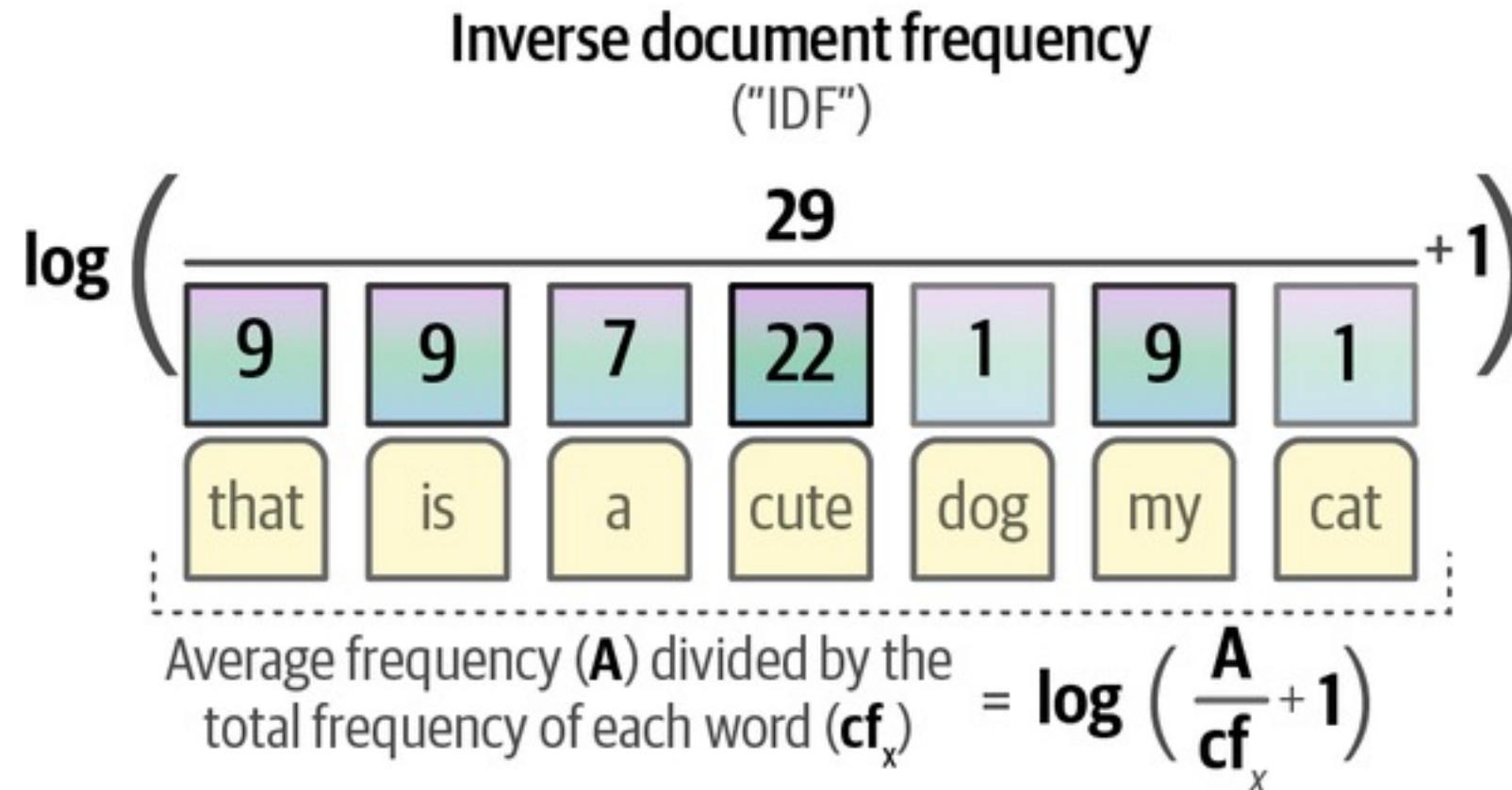
C-TF-IDF

- To represent the topics BERTopic uses a class-based variant of the TF-IDF (**c-TF-IDF**)
- **c-TF**: computes the frequency of terms x within the entire cluster c rather than the document



C-TF-IDF

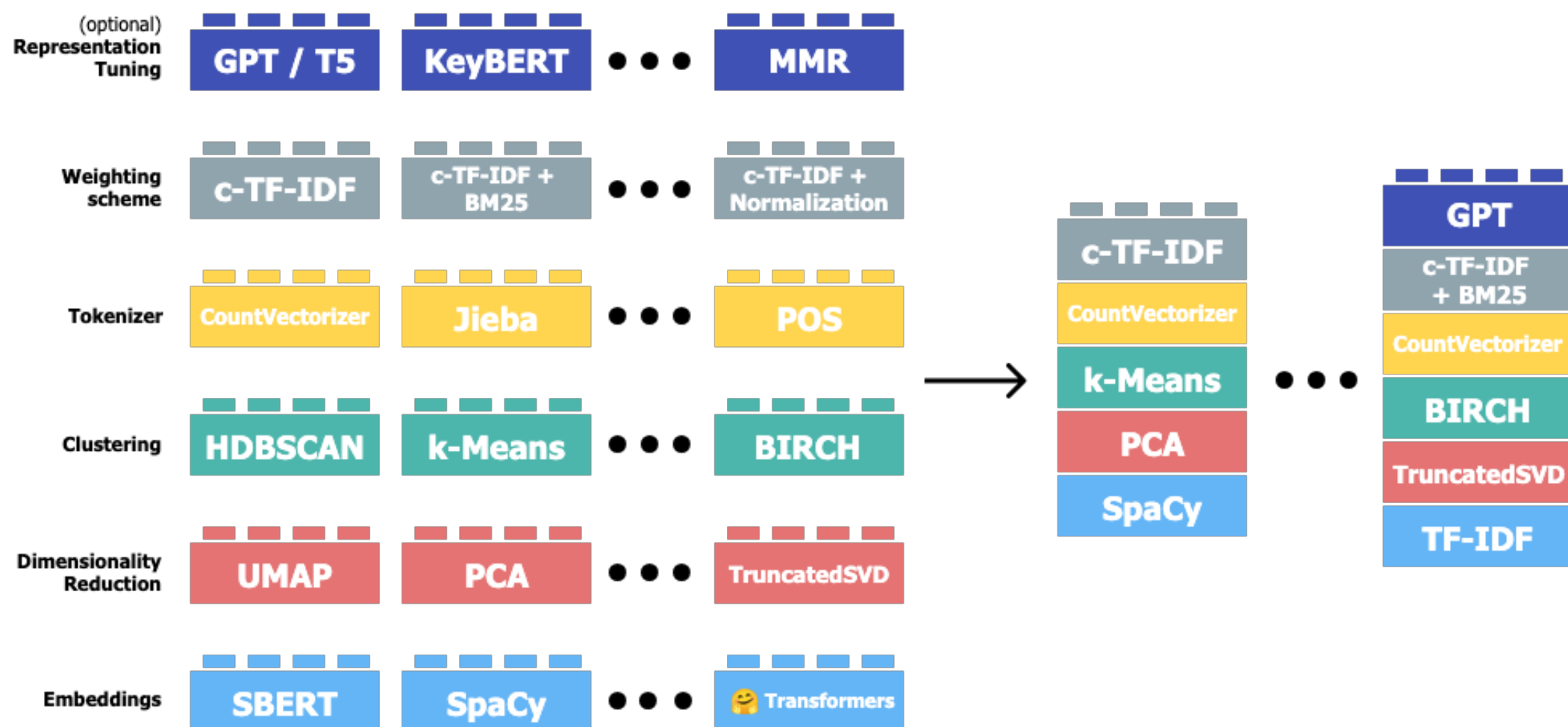
- **IDF**: the main idea is to give more weight to terms x that are meaningful for the cluster and less on the ones that are used in all clusters c



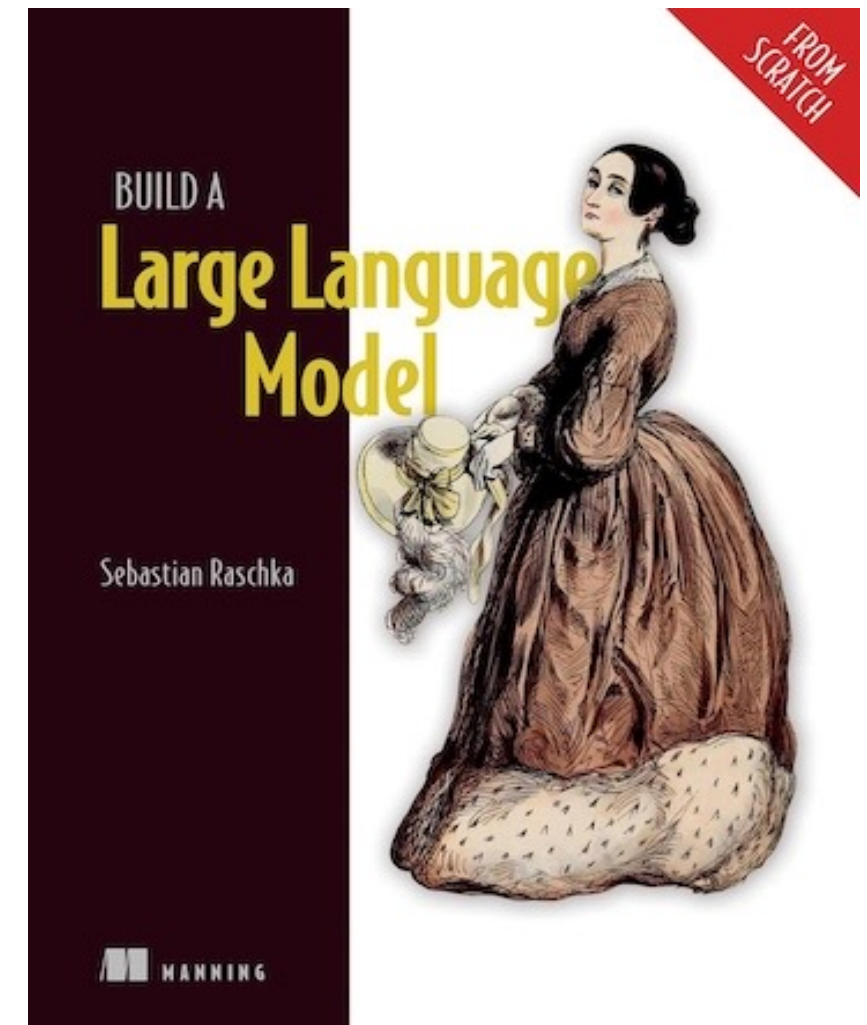
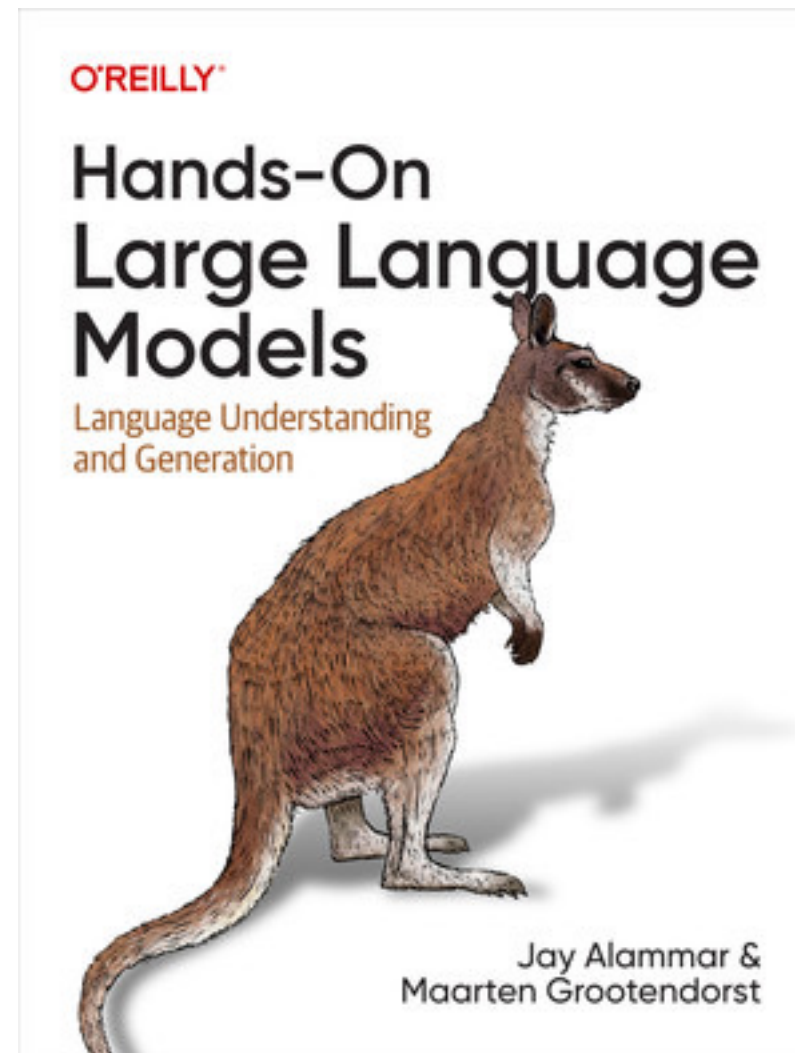
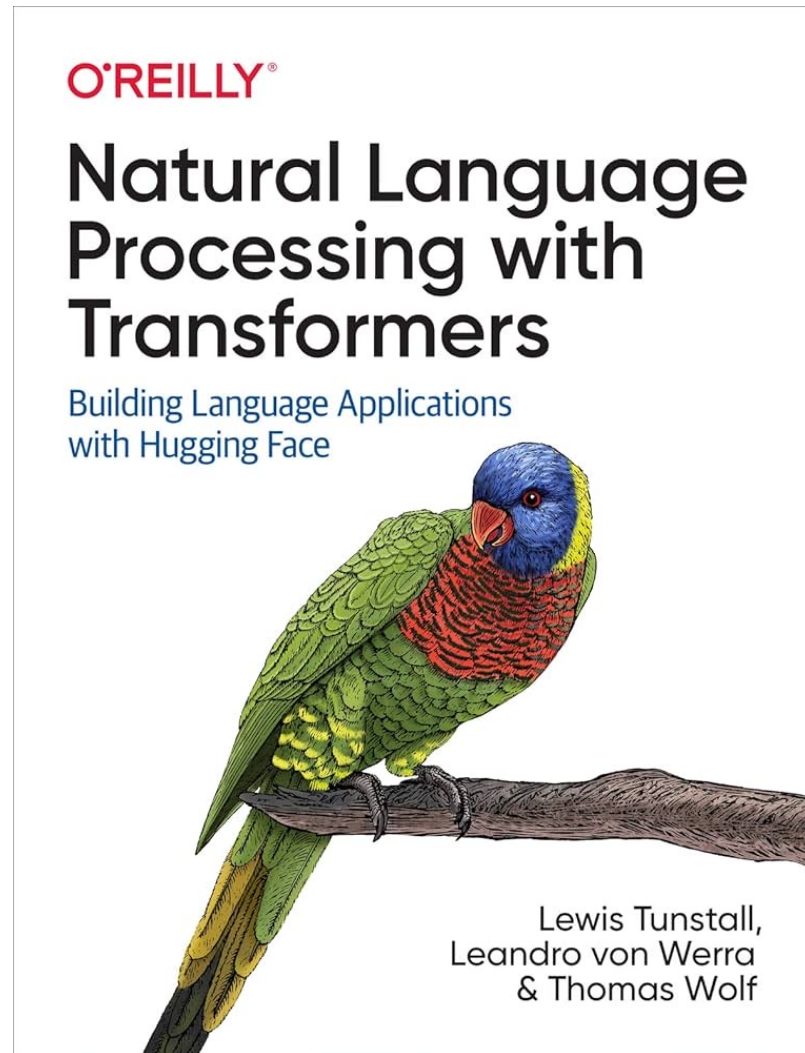
- $c\text{-TF-IDF} = \|tf_{x,c}\| \log \left(\frac{A}{cf_x} + 1 \right)$

BERTOPIC

- Hence, BERTopic biols down to **two steps** that are independent: **clustering** and **topic representation**
- **Lego-block structure**: The independence of the steps allows for a high level of modularity that allows each component to be build with another method
- More details can be found in the [documentation](#)



WANT TO KNOW MORE ON LLMS

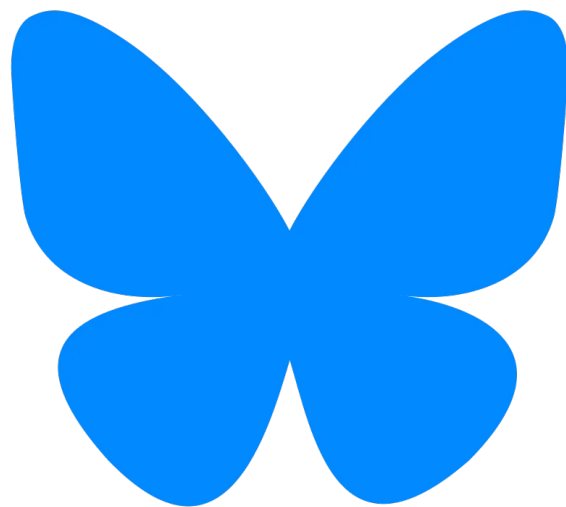


REMEMBER

Post about this lecture:

[#SMWA2026](#)

[#Lecture5](#)



SOCIAL MEDIA AND WEB ANALYTICS

Prof Dr Matthias Bogaert